

実験ログ: プロシージャルモデリング (121~)

2026-09-24 | 収録 42 件

Houdini 研究部で、hython で実際に測った実験の記録。数字はどれも測った値。

収録している実験

- ・ polyexpand2d は角を丸めずに尖らせる — 20° の鋭い角でも。Divisions は距離を等分する
- ・ shrinkwrap は凸包そのもの —

中の点をいくら足しても立方体は体積1.000000、同じ平面の三角形はまとめる

- ・ L-System のコッホ曲線は式どおり — 全長 4^n ・端の間隔 3^n 。小数の世代は途中の形になる
- ・ 点と面の数は式で先に分かる — divide は $n-2$ 、convertline は $V+F-2$ 。edgedivide は既定で点が2倍
- ・ cluster (k-means) は塊を理論の上限どおりに分ける — 重なる塊でも $\Phi(s/\sigma)^2$ と1%以内
- ・ findshortestpath の道のりは Edge の距離と同じ — 四角の網では $|x|+|z|$
- ・ extracttransform は移動と回転だけを取り出す — 拡大は捨てて distortion に出す
- ・ measure の曲率は既定のままだと式と合わない — Divide Element Area

を入れると球・円柱とも0.03%以内

- ・ normal の Cusp Angle は「曲がり角がこれより大きければ角を立てる」 — ちょうど等しいと、なめらか

- ・ bend は長さを保って円弧に曲げる — 端の位置は式と差0、範囲の外は接線の向きにまっすぐ
- ・ attribpromote の11通りのまとめ方 — Median は偶数個なら上の方、Mode は同数なら最小の値
- ・ ray の Direction には最初から @N の式が入っている — Python で set しても変わらず、真下に落ちなかった

- ・ vdbfrompolygons の SDF は帯の中だけ本当の距離（ずれ0.0008以下） — 帯の外は $\pm$ (Band Voxels×ボクセル) で平ら

- ・ attribblur の1回は「となりの平均へ半分近づく」を2度 — ぼけ幅は $\sqrt{\text{回数}}$ 。線では Pin Border で全く動かない

- ・ copytopoints の向きは orient が N に勝つ — N は +z を、up は +y を合わせる。scale と pscale は掛け算

- ・ pointsfromvolume の点は、格子なら1点 s^3 ・四面体なら $s^3/\sqrt{2}$ — 箱では面の上にも並ぶ

- ・ platonic の Radius は、四面体・八面体・二十面体では頂点までの距離 — 立方体は辺が1、十二面体は0.9933

- ・ circle の Polygon は頂点が円周に乗る正 n 角形 — 弧の Divisions は辺の数

- ・ torus の Rows は管の断面、Columns は大きな輪 — 体積は2つの内接多角形の積で決まる

- ・ mirror の継ぎ目は「元と鏡像の距離」が Consolidate Seam 未満ならまとまる — 境目ちょうどはまとまらない

- ・ clip で切った球の面積は $2\pi h$ — 比は切る高さに関係なく、球全体と同じだけずれる

- ・ twist SOP の6つの操作 — 体積を保つのは Shear だけ。Squash は長さ $(1+s)$ 倍・太さ $1/(1+s)$ 倍

- ・ tube の円錐台は式 $\times$ 内接多角形の比 — Radius の1つ目が上、円錐の先は点が重なったまま

- divide の Bricker は形の端 (+ Offset) から Size おきに線を引く — Convex は n 角形を $n-2$ 枚の三角形か、その半分の四角形に
- sphere の Polygon は点 $10f^2+2$ ・面 $20f^2$ の測地球 — 同じ点の数なら Polygon Mesh より体積の不足が約3割小さい
- polybevel の Round は、分割を増やすと「角を丸めた箱」の体積に近づく — 平らな面取りは $1 - 6d^2 + (16/3)d^3$
- polyfill の四角形の塞ぎ方は、辺が奇数の穴を塞がない — 三角形は 1つの穴に $n-2$ 枚、Grid は少しふくらむ
- scatter の density 属性は点を $k/(1+k)$ に分ける — 数を決めないと Density Scale × 面積のあたりで揺れる
- polywire の管は Wire Radius に内接する正 n 角柱で両端に蓋 — 折れ目は Prevent Joint Buckling で $r/\cos(\theta/2)$ に広がる
- uvlayout は長方形 12 枚で升の 48~84% を埋める — Padding は Search Resolution が粗いほど大きく効く
- attribtransfer は Distance Threshold までそのまま運び、Blend Width の外側で $(1 - t^2)^2$ に落とす
- subdivide の crease は「重み w の回数までは尖ったまま」 — $w \geq \text{Iterations}$ なら箱はそのまま、体積 1
- vdbfromparticles の粒は pscale を半径にした球 — 2つ重ねると「2つの球の和」の体積に、升目の2乗で近づく
- smooth の縮み方は $r = 1/(1 + \text{Strength} \cdot L^q / C(2q, q))$ — 36 通りで6桁一致。Filter Quality を上げるほど形を保つ
- smooth の3つの Method は、点が等間隔の円では同じ縮み方 — 開いた線の端は Constrained Boundary で止まり、波の高さは $1/(1 + S \cdot L/2)$ だけ残る
- 点の間隔がそろわない円では、smooth は粗い側を大きく縮める — 3つの Method の差は小さく、どれも丸さを保たない
- polyextrude の Inset は上の面を i だけ内側へ寄せる — 体積は角錐台の式どおり、 $i \geq 0.5$ で四角錐になり、それ以上は裏返らない
- vdbreshapesdf の Dilate ・ Erode は、球の半径を Offset × 升の大きさだけ変える — Iterations は効かず、Open ・ Close は形をほぼ変えない
- vdbreshapesdf の Open は箱の角を r より少し大きく丸める — Close も凸な箱を少しだけ削る
- SDF の帯を 25 升に広げると、vdbreshapesdf の Close は箱をほとんど削らなくなる — Open の丸みはかえって大きくなる
- vdbreshapesdf の Open で丸めた辺の断面は円弧 — 半径は Offset × 升より 2.5~3.2 升大きい
- Open で丸めた辺の半径が約 3 升大きいのは、SDF の帯のせいではない — 帯 3~25 升で足し分は変わらない

polyexpand2d は角を丸めずに尖らせる — 20°の鋭い角でも。Divisions は距離を等分する

2026-09-24

何を作ったか

1辺2の正方形を polyexpand2d で外と内へずらし、できた線の周の長さを測った。外へ d ずらしたとき、角を尖らせる（留め継ぎ）なら $4 \cdot (2+2d)$ 、角を丸めるなら $8 + 2nd$ になるので、周の長さでどちらか分かる。

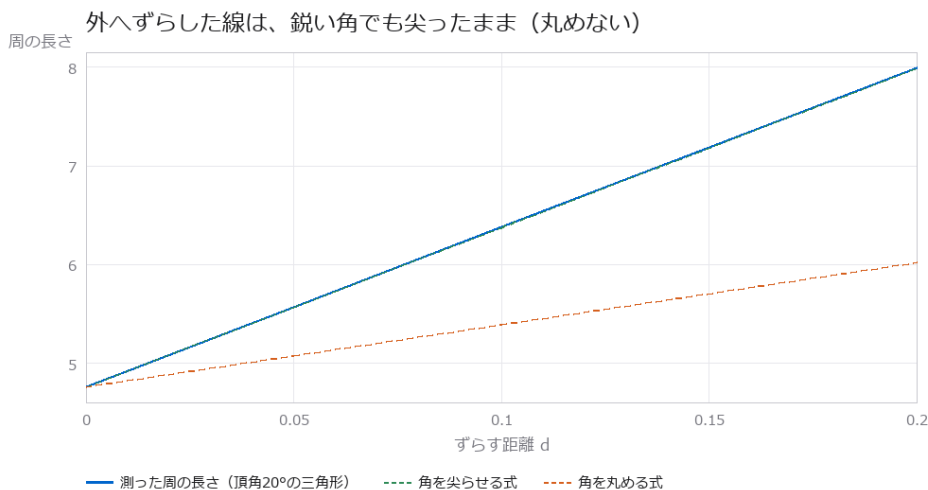
**角は尖らせていた。 ** 3通りの d で、外へずらした周は $4 \cdot (2+2d)$ と一致（差は最大 0.000000）。点の数も4のまま。内へずらした周も $4 \cdot (2-2d)$ とおり。

**頂角20°の鋭い三角形でも、尖らせたまま。 ** 外へずらした線は元と相似な三角形になり、周の長さは (内接円の半径 + d)/(内接円の半径) 倍の式と一致した（差は最大 0.000001）。d = 0.2 だと、先端は 1.152 も外へ伸びる（ $d \div \sin(10^\circ)$ ）。角を丸める道具ではないので、鋭い角の縁取りは先が大きく飛び出す。

**Divisions は、ずらす距離を等分する。 ** Offset 0.2・Divisions 4 にすると、0.2 ずつ4本ではなく、0.05 刻みで 0.05・0.1・0.15・0.2 の4本ができた（内側の周 6.4, 6.8, 7.2, 7.6、外側 8.4, 8.8, 9.2, 9.6）。

正方形（1辺2）をずらしたときの周の長さ

Output = Offset Curves。周の長さは measure の Perimeter。



鋭い三角形でも、測った値は「尖らせる式」に重なる。

d	向き	周の長さ	尖らせる式	丸める式	点
0.1	外	8.800000	8.800000	8.628319	4
0.1	内	7.200000	7.200000	—	4
0.25	外	10.000000	10.000000	9.570796	4
0.25	内	6.000000	6.000000	—	4
0.5	外	12.000000	12.000000	11.141593	4
0.5	内	4.000000	4.000000	—	4

頂角20°の二等辺三角形（高さ2）を外へずらす

尖らせる式 = 元の周 × (r + d)/r (r は内接円の半径)。

d	周の長さ	尖らせる式	丸める式
0.05	5.572494	5.572493	5.081174
0.1	6.377973	6.377972	5.395333
0.2	7.988931	7.988930	6.023651

わかったこと

- ・角は尖らせる。丸い縁取りがほしいときは、別の方法（VDBで太らせて戻すなど）を使う（今回は試していない）。
- ・鋭い角ほど先が伸びる。頂角20°で d=0.2 なら、先端は 1.15 外へ出る。
- ・Divisions は Offset を等分。「0.2 間隔で4本」なら Offset 0.8 ・ Divisions 4 にする。

次にやること

- ・Output を Offset Surfaces にしたときの面積（輪の帯の面積）
- ・へこんだ角（凹の角）を外へずらしたとき、線が重なるか
- ・内へずらしすぎて形が消えるときの振る舞い

shrinkwrap は凸包そのもの — 中の点をいくら足しても立方体は体積1.000000、同じ平面の三角形はまとめる

2026-09-24

何を作ったか

答えが決まる点の置き方で、shrinkwrap
の外形が凸包（へこみの無い、いちばん小さな外形）かを確かめた。

****3D:** 立方体の角8つに、中の点を最大3,000個ほど足しても、外形は立方体のまま。****** 体積は1.000000、正八面体は $4/3$ と一致（差は最大 0.000000）。中の点は外形に1つも使われず、出てきた点は立方体で8個・正八面体で6個だけ。面の数は立方体で6枚（同じ平面の三角形を1枚の四角形にまとめている）、正八面体で8枚。

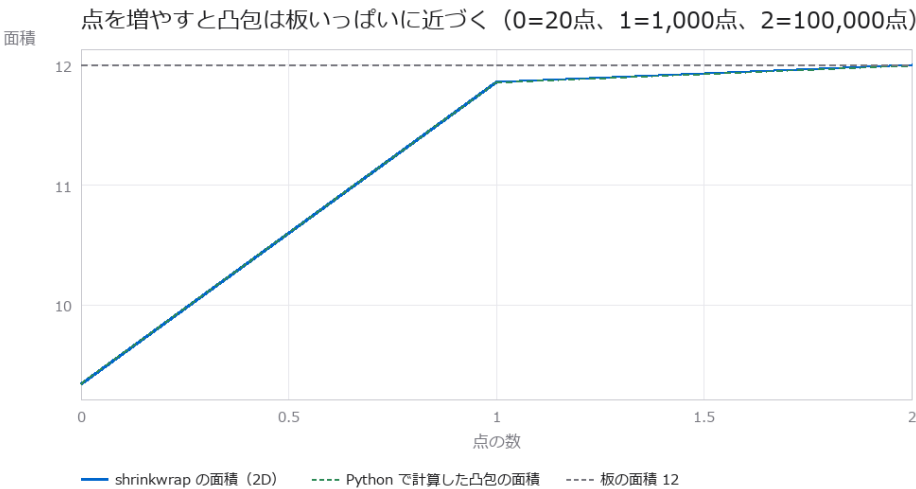
****2D:** scatter した点の外形の面積は、Python で別に計算した凸包の面積と一致******（差は最大 0.000001）。

******ついでに分かったこと: scatter の点は、既定の Relax Iterations で板の縁にぴったり乗る。****** 1,000 点のうち 52 点が、板の縁の上にあった（Relax を切ると 0 点）。shrinkwrap は一直線上の点を外すので、外形の点は 17 個まで減る（縁に乗った点を数えると 61 個）。

速さ: 10万点の 2D で 0.10 秒。

点の置き方と、外形の大きさ

中の点は $|x|+|y|+|z| < 0.49$
の範囲だけに置いた（立方体・正八面体のどちらの内側にも入る）。体積・面積は measure で出した。



shrinkwrap（青）と Python の凸包（緑の破線）は重なる。

点の置き方	入力点	外形点	外形面	体積・面積	式・別計算	秒
立方体の角8つ +中の点	8	8	6	1.000000	1.000000	0.0749
立方体の角8つ +中の点	41	8	6	1.000000	1.000000	0.0129

点の置き方	入力の点	外形の点	外形の面	体積・面積	式・別計算	秒
立方体の角8つ +中の点	3,235	8	6	1.000000	1.000000	0.0136
正八面体の頂点 6つ+中の点	6	6	8	1.333333	1.333333	0.0112
正八面体の頂点 6つ+中の点	39	6	8	1.333333	1.333333	0.0116
正八面体の頂点 6つ+中の点	3,233	6	8	1.333333	1.333333	0.0133
板（4×3）に scatter した点（2D）	20	8	1	9.345696	9.345697	0.0004
板（4×3）に scatter した点（2D）	1,000	17	1	11.858295	11.858296	0.0086
板（4×3）に scatter した点（2D）	100,000	20	1	11.999428	11.999428	0.1012

わかったこと

- ・ shrinkwrap は凸包。中の点は使われず、角の点だけが残る。
- ・ 同じ平面の面はまとめる。立方体の外形は三角形12枚ではなく四角形6枚。
- ・ scatter の点は縁にも乗る。Relax
を入れたまま（既定）だと、1,000点中52点が縁の上。縁から離したいときは Relax
を切るか、少し内側の板に散らす。

次にやること

- ・ shrinkwrap の Shrink Amount を上げたとき、どれだけ内側へ寄るか
- ・ scatter の Relax で縁に乗る点の割合が、点の数でどう変わるか
- ・ 3D の凸包で、点が球面上にあるとき（全部の点が外形に使われる）の速さ

L-System のコッホ曲線は式どおり — 全長 4^n ・端の間隔 3^n 。小数の世代は途中の形になる

2026-09-24

何を作ったか

lssystem に、始まり F、規則 $F \rightarrow F+F--F+F$ 、角度 60° 、1歩の長さ 1 を入れて、コッホ曲線を育てた。n 世代で、線分は 4^n 本、全長は 4^n 、端から端までの距離は 3^n になるはず。

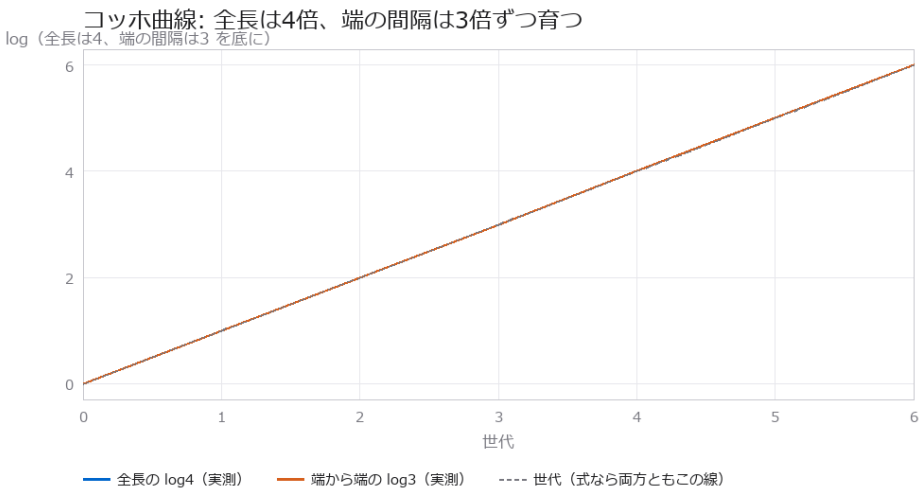
**0~6 世代のすべてで式どおり。 ** 線分数は 4^n 本で一致、端の間隔は 3^n （差は最大 0.000000）、全長のずれは最大 0.5 ppm（6世代で 4096.001953。小数の丸めの範囲）。6世代は 4,097 点を 1.6 ミリ秒で作った。

曲線は y 方向（上）へ伸び、盛り上がりは x 方向に出た。高さは端の間隔の $\sqrt{3}/6$ 倍（6世代で $210.4445 = 729 \times \sqrt{3}/6$ ）。

Generations に 2.5 を入れると、点の数は3世代と同じ 65 点で、全長は 40（2世代 16 と3世代 64 の間）、端の間隔は 21.2942。3世代の形へ向かう途中の形らしい（Continuous Length が既定で入っている。線分1本ずつの長さ、と、切った場合は確かめていない）。

世代と、線分・全長・端の間隔

全長は measure の Perimeter。端の間隔は最初と最後の点の距離。



log を取ると、全長（底4）と端の間隔（底3）はどちらも世代の数に重なる。

世代	点	全長	4^n	端の間隔	3^n	ミリ秒
0	2	1.000000	1	1.000000	1	15.4
1	5	4.000000	4	3.000000	3	0.4
2	17	16.000000	16	9.000000	9	0.3
3	65	64.000000	64	27.000000	27	0.5
4	257	256.000031	256	81.000000	81	0.4
5	1,025	1023.999939	1024	243.000000	243	0.6
6	4,097	4096.001953	4096	729.000000	729	1.6
2.5	65	40.000000	—	21.294228	—	0.4

わかったこと

- ・規則どおりに育つ。線分 4^n 本、全長 4^n 、端の間隔 3^n 。
- ・速い。4,097 点でも 2 ミリ秒。
- ・小数の世代は、次の世代と同じ点の数で、長さは途中の値。
- ・全長 ÷ 端の間隔 = $(4/3)^n$ 。世代を増やすほど、同じ幅に長い線が詰まる（フラクタル）。

次にやること

- ・ Step Size Scale と " 記号で、枝が世代ごとに短くなる木の全長
- ・ Continuous Length を切ったとき、小数の世代がどうなるか
- ・ Tube で出したときの点の数と速さ

点と面の数は式で先に分かる — divide は $n-2$ 、convertline は $V+F-2$ 。edgedivide は既定で点が2倍

2026-09-24

何を作ったか

形を変える箱を通したとき、点と面の数が式どおりになるかを、5つの箱で確かめた。数えるのは intrinsic の pointcount ・ primitivecount ・ vertexcount。

****divide:** n 角形1枚は、三角形 $n-2$ 枚になる******（3～100角形の6通りですべて一致）。

****facet の Unique Points:** 点の数は頂点の数になる******（箱 8→24、格子 25→64、球 242→1,008 で一致）。面ごとに自分の点を持つので、角がくっつきする代わりに点が増える。

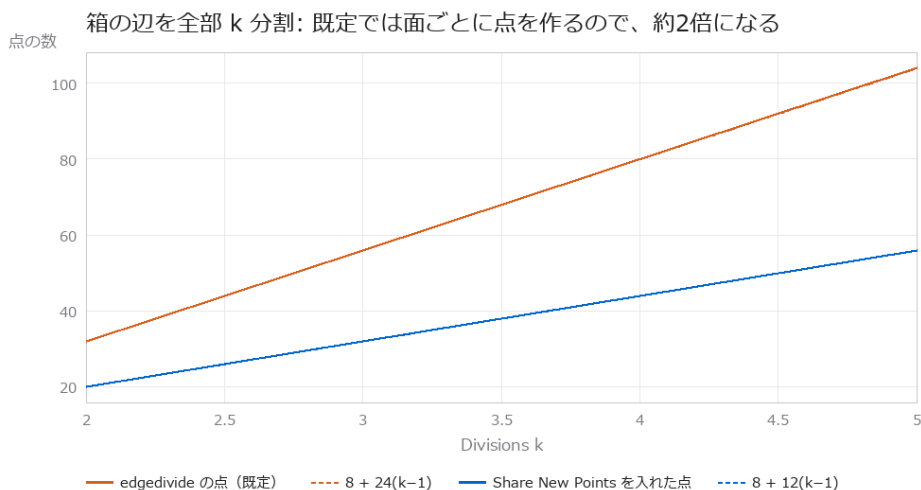
****convertline:** 線の本数は辺の数。****** 閉じた形ではオイラーの式から 辺 = 点 + 面 - 2 で、箱 12 本・球 504 本と一致。開いた格子 5×5 は 40 本（縦20+横20）。

****polysoup:** 面が1つになり、頂点の数が点の数まで減る******（箱 24→8、球 1,008→242）。同じ点を使う頂点をまとめる（Merge Identical Vertices が既定で入っている）。

****edgedivide** は、既定では点が思ったより多い。****** 箱の12本の辺を k 分割すると、点は $8 + 12(k-1)$ ではなく **** $8 + 24(k-1)$ **** になった。隣り合う2つの面が、それぞれ自分の新しい点を作るため。Share New Points を入れると $8 + 12(k-1)$ になる（すべて一致）。

divide (n 角形1枚 → 三角形)

circle を Polygon で n 分割したもの。



edgedivide の既定（橙）は、辺ごとに点を2倍作る。

n	三角形	$n-2$	頂点
3	1	1	3
4	2	2	6
5	3	3	9
8	6	6	18

n	三角形	n-2	頂点
24	22	22	66
100	98	98	294

facet・convertline・polysoup

前 = 元の形。後 = 箱を通したあと。

箱	形	前の点・面・頂点	後の点・面・頂点	式
facet_unique	箱 (8点・6面)	8・6・24	24・6・24	24
facet_unique	格子 5×5 (25点・16面)	25・16・64	64・16・64	64
facet_unique	球 12×24 (242点・2 64面)	242・264・1008	1008・264・1008	1008
convertline	箱 (8点・6面)	8・6・24	8・12・24	12
convertline	格子 5×5 (25点・16面)	25・16・64	25・40・80	—
convertline	球 12×24 (242点・2 64面)	242・264・1008	242・504・1008	504
polysoup	箱 (8点・6面)	8・6・24	8・1・8	—
polysoup	格子 5×5 (25点・16面)	25・16・64	25・1・25	—
polysoup	球 12×24 (242点・2 64面)	242・264・1008	242・1・242	—

edgedivide (箱の12本の辺を全部 k 分割)

k	点 (既定)	8+24(k-1)	点 (Share New Points)	8+12(k-1)
2	32	32	20	20
3	56	56	32	32
5	104	104	56	56

わかったこと

- ・divide は n-2 枚。三角形の数は、割る前に分かる。
- ・facet の Unique Points は、点を頂点の数まで増やす。球 12×24 で約4倍。
- ・閉じた形の辺の数は、点 + 面 - 2。convertline の本数で確かめられる。
- ・edgedivide は既定で辺ごとに点を2つずつ作る。形が繋がったままでもいいなら Share New Points を入れる。

次にやること

- ・edgedivide の既定の出力で、面と面の間に隙間ができていないか (fuse で元に戻るか)
- ・divide の Maximum Edges を 4 にしたとき、n 角形が何枚の四角形になるか
- ・穴のある形 (トーラス) で、オイラーの式が $V - E + F = 0$ になるか

cluster (k-means) は塊を理論の上限どおりに分ける — 重なる塊でも $\Phi(s/\sigma)^2$ と1%以内

2026-09-24

何を作ったか

中心の分かっている4つの塊 ($(\pm s, \pm s)$ に各 2,000 点、正規分布のばらつき σ) を作り、cluster に「4つに分けて」と頼んだ。元の塊の点が、どれだけ同じ番号にまとまるか（純度）を測った。

塊が重なっていると、どんな分け方でも100%は取れない。境目は $x=0 \cdot z=0$

なので、点が正しい側にいる確率は1軸あたり $\Phi(s/\sigma)$ （正規分布の累積）、2軸で $\Phi(s/\sigma)^2$ が上限になる。

**4通りすべてで、純度はこの上限と 0.1% 以内。 ** 十分離れた塊 ($s/\sigma = 10$) は 100%、 $s/\sigma = 1$ まで近づけると 70.7%（上限 70.8%）。k-means は、重なった塊でも取れるだけ取っている。

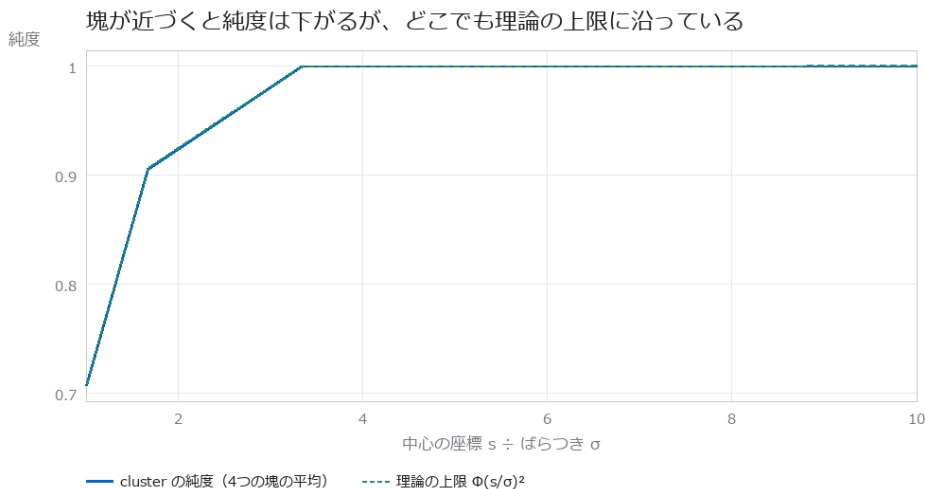
番号ごとの点の平均は、元の中心から最大

0.0066（離れた塊）～0.0793（重なった塊）。結果の番号は点の属性 cluster に入る。

速さ: 8,000 点で 0.06～0.12 秒。

塊の近さと、分け方の純度

純度 = 元の塊ごとに、いちばん多い番号に入った点の割合（4つの平均）。



実測（青）は理論の上限（緑の破線）に沿う。

s	σ	s/σ	番号の数	純度 (4つ)	上限 $\Phi(s/\sigma)^2$	中心のずれ	秒
2	0.2	10.00	4	1.000 / 1.000 / 1.000 / 1.000	1.0000	0.0066	0.119
1	0.3	3.33	4	1.000 / 0.999 / 1.000 / 1.000	0.9991	0.0094	0.058

s	σ	s/σ	番号の数	純度 (4つ)	上限 $\Phi(s/\sigma)^2$	中心のずれ	秒
0.5	0.3	1.67	4	0.915 / 0.898 / 0.898 / 0.910	0.9067	0.0210	0.060
0.3	0.3	1.00	4	0.712 / 0.702 / 0.688 / 0.728	0.7079	0.0793	0.060

わかったこと

- ・離れた塊は100%正しく分ける。
- ・重なった塊でも、理論の上限まで取る。 $s/\sigma = 1$ で約70%。それ以上は、どの方法でも無理。
- ・番号は cluster 属性。色分けや、塊ごとの処理にそのまま使える。

次にやること

- ・塊の大きさ（点の数）が違うとき、k-means が大きな塊を割ってしまうか
- ・num_clusters を実際の塊の数と違う数にしたときの分かれ方
- ・clusterpoints との結果と速さの比較

findshortestpath の道のりは Edge の距離と同じ — 四角の網では $|x|+|z|$

2026-09-24

何を作ったか

実験119 で、distancealonggeometry の Edge

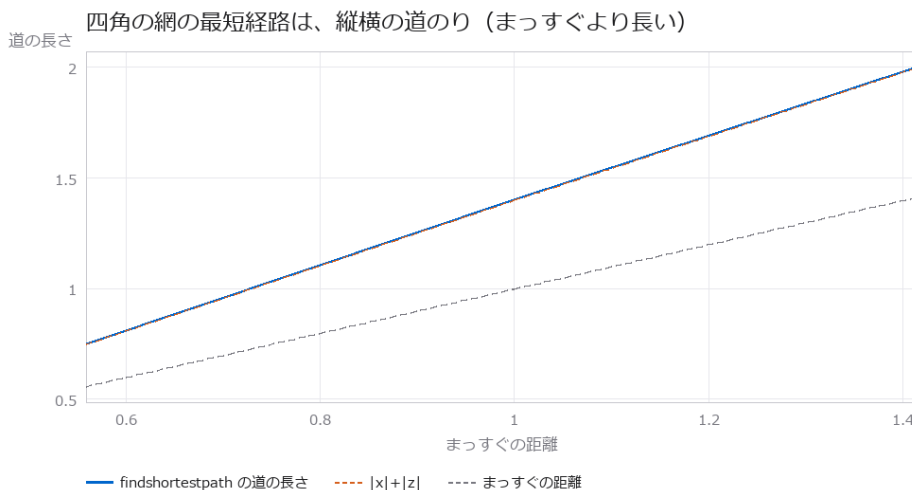
は「辺をたどった道のり」だと分かった。findshortestpath の道の長さも同じになるかを確認めた。2×2 の四角の網（41×41点）の真ん中から、4つの点へ道を引いた。

4本すべてで、cost・道の線の長さ・Edge の距離・ $|x|+|z|$ の4つが完全に一致。たとえば (1, 1) までは 2.0（まっすぐなら $\sqrt{2} \approx 1.414$ ）。道は縦横の辺だけを通る。

道の長さは点の属性 cost にも入る。4本の道を出すのに 85 ミリ秒。

行き先ごとの道の長さ

Output Paths = From any start to each end。道の長さは measure の Perimeter。



道の長さは $|x|+|z|$ の線に乗り、まっすぐの距離より長い。

行き先	道の点	道の長さ	cost	Edge の距離	$ x + z $	まっすぐ
(0.5, 0.25)	16	0.750000	0.750000	0.750000	0.750000	0.559017
(-0.8, 0.6)	29	1.400000	1.400000	1.400000	1.400000	1.000000
(1, 1)	41	2.000000	2.000000	2.000000	2.000000	1.414214

わかったこと

- ・ findshortestpath ・ Edge ・ attribfill の到着時間は、同じ「辺の道のり」。（実験115・119 とあわせて）

- ・ 四角の網では、斜めの行き先ほど遠回りになる。45°の向きで約1.41倍。

- ・ まっすぐに近い道がほしいなら、網を三角形や remesh にする。（実験119 では remesh で Edge のずれが約1/6 になった）

次にやること

- ・ 三角の網・ remesh した網で、道が斜めに通るか
- ・ コスト属性（坂の急さ）を付けて、山を避ける道になるか

extracttransform は移動と回転だけを取り出す — 拡大は捨てて distortion に出す

2026-09-24

何を作ったか

半径1の球に、分かっている移動・回転・拡大をかけ、元の形と変形後の形を extracttransform に渡した。取り出した変形を元の形にかけ直し、変形後の形とのずれを測った。

****出てくるのは、点1つと4つの属性だけ:****

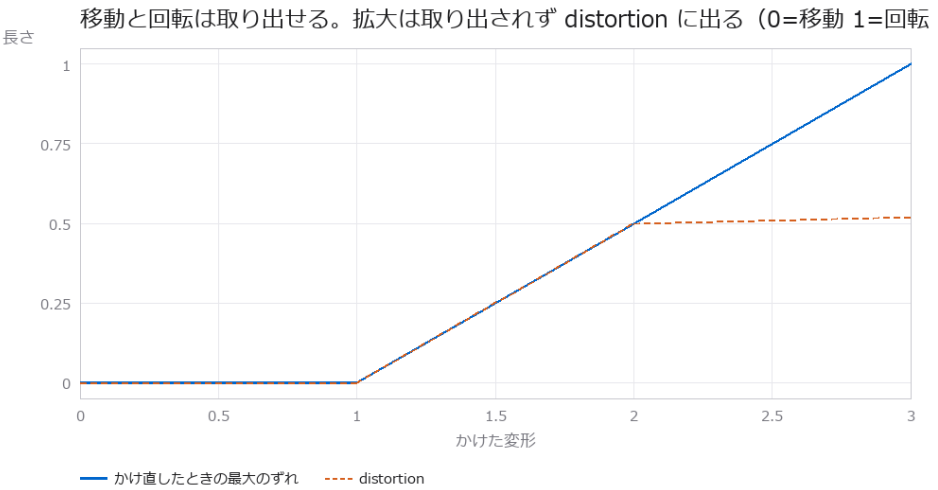
P（変形後の中心）、pivot（元の中心）、orient（回転の四元数）、distortion。拡大の属性は無い。

****移動と回転は、ぴったり取り出せた。**** かけ直したずれは、移動だけ・回転だけのどちらも 0.000000。回転 (10°, 20°, 30°) は、1つの軸のまわりの 35.8171° として出た (xyz の順に掛けた回転行列から計算すると 35.8171°)。P は、元の中心を変形した位置と一致（差は最大 0.000000）。

****拡大は取り出されない。**** 1.5 倍にすると、かけ直しのずれは 0.5（半径 1 → 1.5 の差）で、distortion も 0.5。x だけ2倍に伸ばすと、ずれ 1・distortion 0.5203。distortion は「かたい変形では説明できない分」の大きさの目安になる（伸ばしたときに最大のずれと一致しない理由は確かめていない）。

かけた変形と、取り出した結果

元の形は半径1の球（中心 (0.3, 0.1, -0.2)）。transform SOP (xform) で変形。かけ直し = (元の点 - pivot) × orient の回転 + P。



拡大を含むと、かけ直してもずれが残り、distortion が0でなくなる。

変形	移動	回転	拡大	取り出した角度	かけ直しのずれ	distortion
move	(1.0, 2.0, 3.0)	(0, 0, 0)	(1, 1, 1)	0.0000°	0.000000	0.000000
turn	(0, 0, 0)	(10, 20, 30)	(1, 1, 1)	35.8171°	0.000000	0.000000
all	(1.0, -0.5, 2.0)	(10, 20, 30)	(1.5, 1.5, 1.5)	35.8171°	0.500000	0.500000
stretch	(0, 0, 0)	(0, 30, 0)	(2.0, 1.0, 1.0)	30.0000°	1.000000	0.520266

わかったこと

- ・ 移動と回転は正確に取り出せる。破片のシミュレーション結果を、軽い変形として持ち運ぶのに使える。
- ・ 拡大は取り出さない。大きさが変わる動きは、distortion が0でなくなることで気づける。
- ・ 回転は1本の軸のまわりの角度として出る。(10°, 20°, 30°) は 35.8° になる。

次にやること

- ・ Use Piece Attribute で、破片ごとに別の変形を取り出せるか
- ・ 点に少しノイズを足したとき、回転の取り出しがどれだけ乱れるか
- ・ 取り出した orient と P を、copy to points でかけ直す手順

measure の曲率は既定のままだと式と合わない — Divide Element Area を入れると球・円柱とも0.03%以内

2026-09-24

何を作ったか

半径 r の球（ガウス曲率 $1/r^2$ ・平均曲率 $1/r$ ）と円柱（ガウス曲率 0 ・平均曲率 $1/(2r)$ ）で、measure の Curvature を式と比べた。極や縁の近くは除いた。

既定のままだと、式の値は出てこない。 半径1の球で、ガウス曲率が 0.000934 （式は 1 ）、平均曲率が 0.030485 。しかも半径2の球でも同じ値（ 0.000934 ）で、大きさに左右されない。網を細かくすると値が小さくなることから、既定では「点のまわりの面積の分を足し込んだ量」を大きさに正規化して出していると考えられる（Divide Element Area が切、Scale Normalize が入。中の計算そのものは確かめていない）。

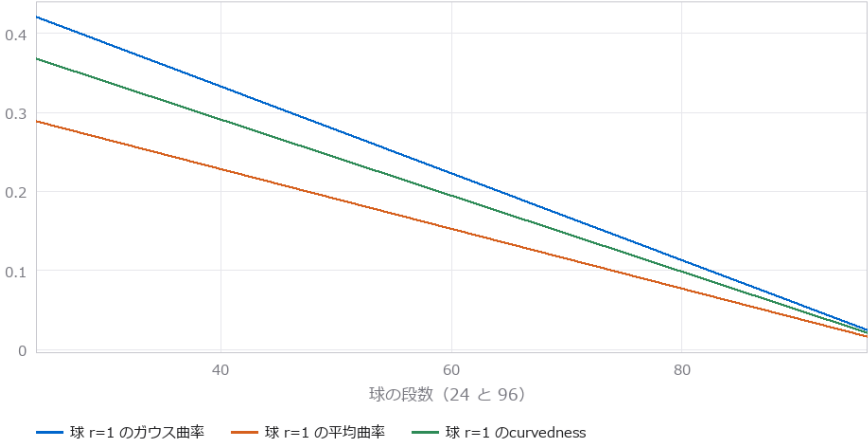
Divide Element Area を入れ、Scale Normalize を切ると、式どおりの曲率になった。 段数24 で最大 0.42% 、段数96 で最大 0.025% のずれ。網を4倍細かくすると、ずれは約 $1/16$ （細かさの2乗で減る）。円柱のガウス曲率は、どちらでも 0 。

平均曲率の符号は、外向きの法線の球でプラスだった。

Divide Element Area 入・Scale Normalize 切のときの曲率（点の平均）

球は polymesh（段数 $\times$ 2倍の列）。 $|y| < 0.8r$ の点だけ。円柱は高さ4、縁から 0.5 以上離れた点だけ。

Divide Element Area を入れれば、曲率は式どおり。網を4倍細かくするとずれは約式とのずれ (%)



どの曲率も、網を細かくすると式に近づく。

形	r	段数	ガウス	式	平均	式	curvednes s	式
球	0.5	24	4.016812	4.0000	2.005776	2.0000	2.007351	2.0000
球	0.5	96	4.000984	4.0000	2.000334	2.0000	2.000422	2.0000
球	1	24	1.004203	1.0000	1.002888	1.0000	1.003676	1.0000

形	r	段数	ガウス	式	平均	式	curvedness	式
球	1	96	1.000246	1.0000	1.000167	1.0000	1.000211	1.0000
球	2	24	0.251051	0.2500	0.501444	0.5000	0.501838	0.5000
球	2	96	0.250062	0.2500	0.500083	0.5000	0.500105	0.5000
円柱	0.5	24	-0.000000	0.0000	1.002862	1.0000	1.418260	1.4142
円柱	0.5	96	-0.000000	0.0000	1.000179	1.0000	1.414466	1.4142
円柱	1	24	-0.000000	0.0000	0.501431	0.5000	0.709130	0.7071
円柱	1	96	-0.000000	0.0000	0.500089	0.5000	0.707233	0.7071

既定のまま (Divide Element Area 切・Scale Normalize 入)

大きさを変えても同じ値。網を細かくすると小さくなる。

形	r	段数	ガウス	平均	curvedness
球	0.5	24	0.015274	0.123325	0.123416
球	0.5	96	0.000934	0.030485	0.030486
球	1	24	0.015274	0.123325	0.123416
球	1	96	0.000934	0.030485	0.030486
球	2	24	0.015274	0.123325	0.123416
球	2	96	0.000934	0.030485	0.030486
円柱	0.5	24	-0.000000	0.181159	0.256197
円柱	0.5	96	-0.000000	0.090458	0.127927
円柱	1	24	-0.000000	0.128099	0.181159
円柱	1	96	-0.000000	0.063964	0.090458

わかったこと

- ・曲率の値そのものがほしいなら、Divide Element Area を入れて Scale Normalize を切る。
- ・既定の値は、網の細かさで変わる。マスクに使うなら、同じ網どうしで比べる。
- ・ずれは網の細かさの2乗で減る。段数96の球で 0.03% 以内。

次にやること

- ・Principal（最小・最大の主曲率）とその向きが、円柱で軸方向と周方向に出るか
- ・Scale Normalize だけを切ったとき、何で割っているのか
- ・トーラスの外側（ガウス曲率がプラス）と内側（マイナス）の符号

normal の Cusp Angle

は「曲がり角がこれより大きければ角を立てる」 — ちょうど等しいと、なめらか

2026-09-24

何を作ったか

k 角柱の側面では、隣り合う面の向きが 360/k 度ずつ変わる。normal で頂点ごとの法線を作り、Cusp Angle を振って、側面の角の点で法線が分かれる（角がくっきりする）かを数えた。

35 通りのうち 35 通りで、「曲がり角 > Cusp Angle なら分かれる」とおりだった。

4・6・8・12・24 角柱のすべてで、境目は曲がり角そのもの。既定の 60°

なら、6角柱（60°）までは丸く、4角柱（90°）は角が立つ。

ちょうど等しいときは、分かれる（なめらか）。 さらに、Cusp Angle

を曲がり角より少しだけ小さくしても、まだ分かれる幅があった: 90° では 0.05°

小さくしても分かれず 0.1° で分かれ、45° では 0.03°/0.05°、15° では

0.01°/0.03°。どれも曲がり角の 0.1~0.2%

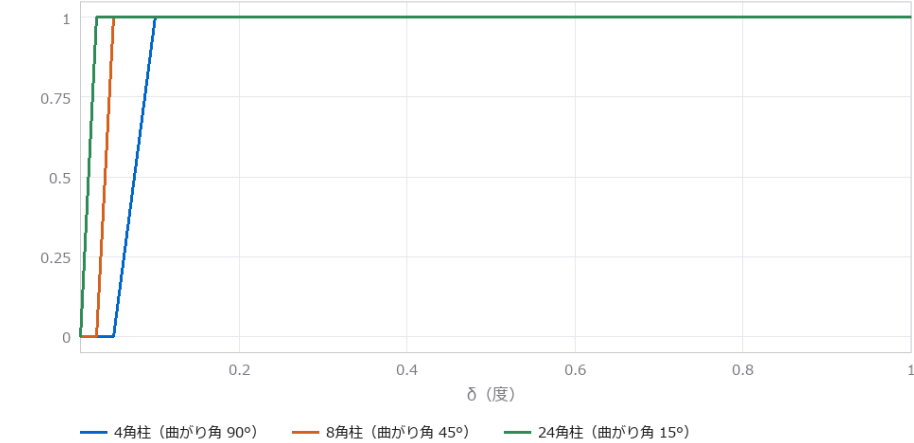
ほどの幅で、計算の誤差を見込んだ余裕と考えられる（中の計算は確かめていない）。

角柱の曲がり角と Cusp Angle

Add Normals to =

Vertices。側面の中ほどの点で、頂点ごとの法線が2つ以上あるものを「分かれた」とした。

Cusp Angle を曲がり角より δ だけ小さくしたとき、法線が分かれるか（1=分かれる、0=分かれず）



曲がり角のすぐ下では、まだ分かれる。少し離れると全部の点で分かれる。

角柱	曲がり角	Cusp Angle	分かれた点の割合	式（曲がり角 > Cusp）
4	90°	10°	1.00	分かれる
4	90°	20°	1.00	分かれる
4	90°	30°	1.00	分かれる
4	90°	45°	1.00	分かれる
4	90°	60°	1.00	分かれる

角柱	曲がり角	Cusp Angle	分かれた点の割合	式（曲がり角 > Cusp）
4	90°	90°	0.00	なめらか
4	90°	90°	0.00	なめらか
6	60°	10°	1.00	分かれる
6	60°	20°	1.00	分かれる
6	60°	30°	1.00	分かれる
6	60°	45°	1.00	分かれる
6	60°	60°	0.00	なめらか
6	60°	90°	0.00	なめらか
6	60°	60°	0.00	なめらか
8	45°	10°	1.00	分かれる
8	45°	20°	1.00	分かれる
8	45°	30°	1.00	分かれる
8	45°	45°	0.00	なめらか
8	45°	60°	0.00	なめらか
8	45°	90°	0.00	なめらか
8	45°	45°	0.00	なめらか
12	30°	10°	1.00	分かれる
12	30°	20°	1.00	分かれる
12	30°	30°	0.00	なめらか
12	30°	45°	0.00	なめらか
12	30°	60°	0.00	なめらか
12	30°	90°	0.00	なめらか
12	30°	30°	0.00	なめらか
24	15°	10°	1.00	分かれる
24	15°	20°	0.00	なめらか
24	15°	30°	0.00	なめらか
24	15°	45°	0.00	なめらか
24	15°	60°	0.00	なめらか
24	15°	90°	0.00	なめらか
24	15°	15°	0.00	なめらか

わかったこと

- ・ Cusp Angle は「角と見なす曲がり」のしきい値。曲がり角がこれより大きい所だけ角が立つ。
- ・ 既定の 60° では、6角柱までは丸く見える。六角ナットの角を立てたいなら 60° 未満にする。
- ・ しきい値ちょうどの形は、丸いほうに倒れる。わずかに小さくしても同じ。確実に角を立てるなら、1° 以上下げる。

次にやること

- Weighting Method (均等・角度・面積) で、なめらかな法線の向きがどう変わるか
- 箱 (90°) の角を、Cusp Angle を変えずに丸く見せる方法 (面取り)

bend は長さを保って円弧に曲げる — 端の位置は式と差0、範囲の外は接線の向きにまっすぐ

2026-09-24

何を作ったか

z 方向に長さ1の線（100分割）を、bend の捕まえる範囲（origin 0・dir z・length 1）いっぱい角度 θ だけ曲げた。長さを保って円弧になるなら、半径は $R = 1/\theta$ （ラジアン）で、範囲の終わりの点は $y = R(1 - \cos \theta)$ 、 $z = R \sin \theta$ に来るはず。

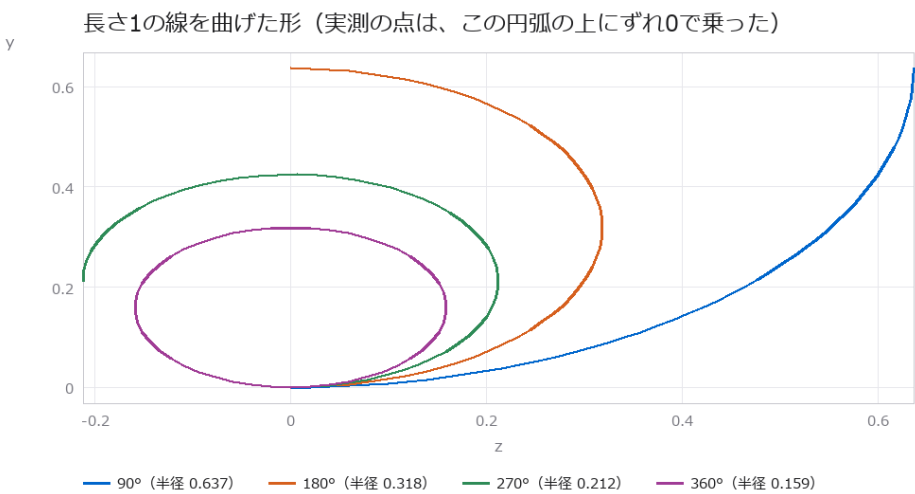
****45°～360° の5通りで、範囲の終わりの点は式と一致****（ずれ最大 0.000000）。360°では、線がちょうど1周して始まりの点に戻った。

****範囲の外（長さ2の線の後ろ半分）は、曲がり終わりの接線の向きにまっすぐ伸びた。****
端の点も式どおり（ずれ 0.000000）。180° なら後ろ半分は真後ろ（-z）へ向かう。

全長は少しだけ短くなる（360° で 0.999835）。これは点が円弧の上に乗ったまま、点と点の間がまっすぐな弦になるため。弦の長さの式 $\sin(h)/h$ （h は1区間の中心角の半分）と比べると、ずれは最大 0.000001。点そのものは円弧の上にある。

曲げる角度と、点の位置・全長

Limit Deformation は既定（入）。曲げる向きは y（既定の up）。



曲げる角度ごとの円弧。半径は $1/\theta$ で、長さはどれも1。

線の長さ	角度	範囲の終わりの点	式	端の点	全長	弦の式
1	45°	(0.372923, 0.900316)	(0.372923, 0.900316)	(0.372923, 0.900316)	0.999997	0.999997
1	90°	(0.63662, 0.63662)	(0.63662, 0.63662)	(0.63662, 0.63662)	0.999990	0.999990
1	180°	(0.63662, -0)	(0.63662, 0)	(0.63662, -0)	0.999959	0.999959
1	270°	(0.212207, -0.212207)	(0.212207, -0.212207)	(0.212207, -0.212207)	0.999907	0.999907

線の長さ	角度	範囲の終わりの点	式	端の点	全長	弦の式
1	360°	(0, 0)	(0, -0)	(0, 0)	0.999835	0.999836
2	45°	(0.372923, 0.900316)	(0.372923, 0.900316)	(1.08003, 1.60742)	1.999997	1.999997
2	90°	(0.63662, 0.63662)	(0.63662, 0.63662)	(1.63662, 0.63662)	1.999990	1.999990
2	180°	(0.63662, -0)	(0.63662, 0)	(0.63662, -1)	1.999959	1.999959
2	270°	(0.212207, -0.212207)	(0.212207, -0.212207)	(-0.787793, -0.212207)	1.999907	1.999907
2	360°	(0, 0)	(0, -0)	(0, 1)	1.999835	1.999836

わかったこと

- bend は長さを保つ円弧。半径は「範囲の長さ ÷ 角度（ラジアン）」。
- 範囲の外はまっすぐ付いてくる。曲がり終わりの向きに、そのまま伸びる。
- 全長の不足は弦の分だけ。100分割で 360° 曲げても 0.02% 以内。

次にやること

- Twist と Taper を同時に入れたときの形
- 範囲の途中から始まる線（origin をずらす）で、手前がどうなるか
- Preserve Volume を入れた Length Scale の体積

attribpromote の11通りのまとめ方 — Median は偶数個なら上の方、Mode は同数なら最小の値

2026-09-24

何を作ったか

格子 4×4 の16点に、 $v = (\text{点番号} \times 7) \bmod 10$ という決まった値を入れ、attribpromote で点 → 全体 (Detail) に上げた。Python で同じ計算をして、まとめ方ごとに比べた。

****最大・最小・平均・合計・2乗の合計・RMS・最初・最後の8通りは、式とすべて一致。平均 4.375、RMS 5.244044 など。**

****Median は、値が偶数個のとき真ん中の2つの平均ではなく、大きい方を返した**** (小さい順に並べた 8番目 4 と 9番目 5 のうち 5。平均なら 4.5) 。

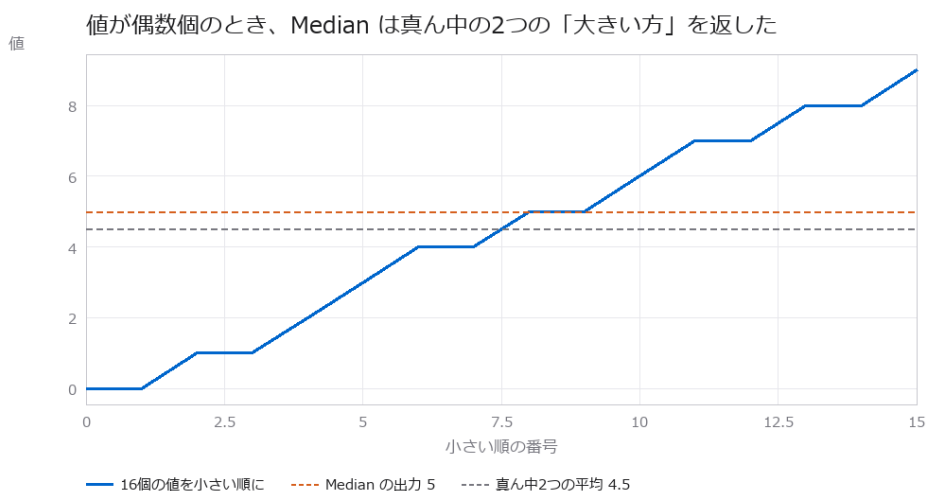
****Mode は、いちばん多く出た値が同数で並ぶとき、その中の最小の値を返した。元の並べ方 (候補 0, 1, 4, 5, 7, 8 →**

0) に加え、最初に出る値が最小にならない並べ方2通りでも確かめた (最初に出てきた値ではない) : 候補 0, 1, 3, 4, 7, 8 → 0 (最初の値は 3) 、候補 1, 2, 4, 5, 8, 9 → 1 (最初の値は 9) 。

点 → 面の Average は、面の4つの角の値の平均と一致した (ずれ 0) 。

点 → 全体へのまとめ方 (16点)

値は 0, 7, 4, 1, 8, 5, 2, 9, 6, 3, 0, 7, 4, 1, 8, 5 (点番号の順) 。



偶数個の Median は、真ん中の2つの上の方 (5) 。平均の 4.5 ではない。

まとめ方	出た値	式
Maximum	9	9
Minimum	0	0
Average	4.375	4.375
Sum	70	70
Sum of Squares	440	440
Root Mean Square	5.24404	5.24404

まとめ方	出た値	式
First Match	0	0
Last Match	5	5
Median	5	平均なら 4.5／下 4／上 5
Mode	0	同数の候補 0, 1, 4, 5, 7, 8

わかったこと

- ・ Median は偶数個で上の方。中央値として平均がほしいなら、自分で計算する。
- ・ Mode の同数は最小の値。ばらつきが大きいデータの Mode は当てににくい。
- ・ 点 → 面の Average は、角の平均そのもの。

次にやること

- ・ Piece Attribute を使って、塊ごとにまとめたときの結果
- ・ 頂点 → 点（Vertex → Point）で、共有された点の値がどうまとまるか

ray の Direction には最初から @N の式が入っている — Python で set しても変わらず、真下に落ちなかった

2026-09-24

何を作ったか

半径1の球の上に格子（31×31点、高さ2）を置き、ray で球に落とした。真下へ落とせば、当たった点は $y = \sqrt{1 - x^2 - z^2}$ に来るはず。

****最初は1点も動かなかった。*** 調べると、ray の Direction (dirx・diry・dirz) には最初から @N.x, @N.y, @N.z という式が入っていた。Python で `parmTuple("dir").set((0, -1, 0))` としても、式が残ったままで値は (0.0, 0.0, 0.0) のまま。格子には法線が無いので、光線は格子の向き（上）へ飛び、下の球には当たらなかった。****式を消してから（deleteAllKeyframes）入れると、正しく真下へ落ちた。***（画面で値を打ち込んだときに式が置き換わるかは、確かめていない。格子を球の下に置くと、同じ設定のまま下の面に当たったので、光線が上へ飛んでいたことは確か）

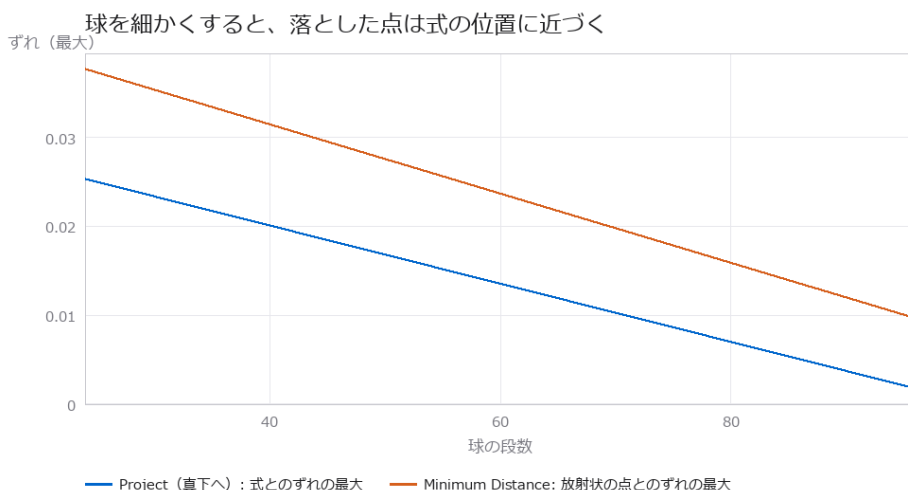
****真下へ落とした点は、式の位置と一致した****（段数96でずれ最大 0.0018、平均 0.00021）。ずれは球が多角形である分で、段数24では最大 0.0253。球の外側を通る光線の点（40点）は、元の高さのまま残った。

****Minimum Distance は、いちばん近い面の上へ動かす。***

中心から放射状に落とした位置とのずれは、段数96で最大 0.0096。多角形の平らな面の上では「いちばん近い点」と「放射状の点」が少し違うため、Project より大きい。

落とし方と、式とのずれ

Project の式は $y = \sqrt{1 - x^2 - z^2}$ 。Minimum Distance は 点 ÷ 中心からの距離（放射状）と比べた。当たった点の中心からの距離も出した（1 なら球の上）。



どちらも、球を細かくするとずれが減る。

球の段数	落とし方	当たった点	外れて残った点	ずれの最大	ずれの平均	中心からの距離 (最小)
24	Project (真下)	921	40	0.025300	0.003635	0.995734
24	Minimum Distance	961	0	0.037646	0.019475	0.997394
96	Project (真下)	921	40	0.001783	0.000209	0.999754
96	Minimum Distance	961	0	0.009621	0.004528	0.999854

わかったこと

- ・ ray の Direction は、最初は @N の式。スクリプトで向きを入れるときは、先に deleteAllKeyframes() で式を消す。
- ・ 法線の無い形から飛ばすと、面の向き（格子なら上）へ飛ぶ。落としたい向きを必ず決める。
- ・ 外れた点は動かない。外れた点を消したいときは、別を選んで消す。

次にやること

- ・ Scale と Lift で、当たった位置から少し戻す・浮かせるときの距離
- ・ ほかにも「最初から式が入っている」つまみがどれだけあるか（node_dump で数える）

vdbfrompolygons の SDF

は帯の中だけ本当の距離（ずれ0.0008以下） —
帯の外は ±（Band Voxels×ボクセル）で平ら

2026-09-24

何を作ったか

半径1の球（polymesh 200×200）から vdbfrompolygons で SDF
を作り、中心から斜めに伸ばした線の上で volumesample した値を、本当の符号付き距離 $|p| - 1$
と比べた。

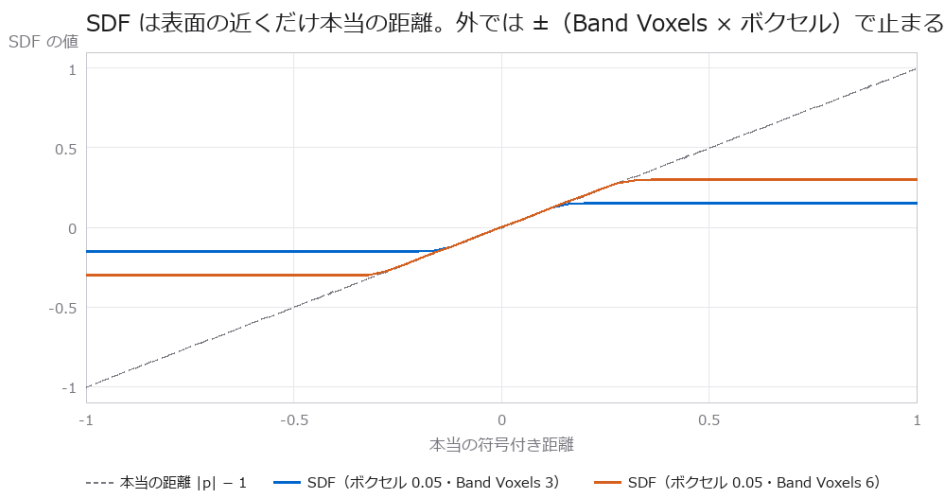
****表面の近くの帯の中では、SDF は本当の距離と一致した****（4通りでずれ最大 0.0008）。ボクセル
0.02 なら 0.0002 まで小さくなる。

****帯の外では値が一定になる。* 外側は +（Band Voxels × ボクセル）、内側は
-（同じ）で平らになった:** ボクセル 0.05・Band Voxels 3 で ±0.15、ボクセル 0.05・Band Voxels 6
で ±0.3、ボクセル 0.02・Band Voxels 3 で ±0.06、ボクセル 0.02・Band Voxels 6 で
±0.12。帯の幅は、Exterior / Interior Band Voxels（ボクセルの数）× ボクセルの大きさ。

つまり、表面から帯の幅より遠い所では「どれだけ遠いか」の情報は無い。SDF
で表面からの距離を使ったマスクを作るときは、帯を必要な距離まで広げておく。

ボクセル・帯の幅と、SDF のずれ

帯の中 = 表面からの距離が（帯の幅 - 1ボクセル）より近い所。帯の外 = （帯の幅 +
1ボクセル）より遠い所。



SDF（実線）は、表面のまわりだけ本当の距離（破線）に重なり、外では平らになる。

ボクセル	Band Voxels	帯の幅	帯の中のずれ（最大）	帯の外の値	作る秒
0.05	3	0.15	0.000708	-0.15, 0.15	0.046
0.05	6	0.3	0.000809	-0.3, 0.3	0.029
0.02	3	0.06	0.000201	-0.06, 0.06	0.032
0.02	6	0.12	0.000201	-0.12, 0.12	0.054

わかったこと

- ・ 帯の中は本当の距離。ずれは 0.001 未満。
- ・ 帯の外は ±帯の幅で止まる。遠くの距離を使いたいなら、Band Voxels を増やす。
- ・ 帯の幅 = Band Voxels × ボクセル。ボクセルを細かくすると、同じ Band Voxels でも帯は細くなる。

次にやること

- ・ Fill Interior を入れたとき、中の値がどうなるか
- ・ vdbsmoothsdf の後でも距離の性質が保たれるか

attribblur

の1回は「となりの平均へ半分近づく」を2度 ー
ぼけ幅は $\sqrt{\text{回数}}$ 。線では Pin Border
で全く動かない

2026-09-24

何を作ったか

401点の線（点の間隔 0.01）に段差の値（左 0・右 1）を入れて attribblur にかける、Python
で同じ計算をくり返したものと1点ずつ比べた。

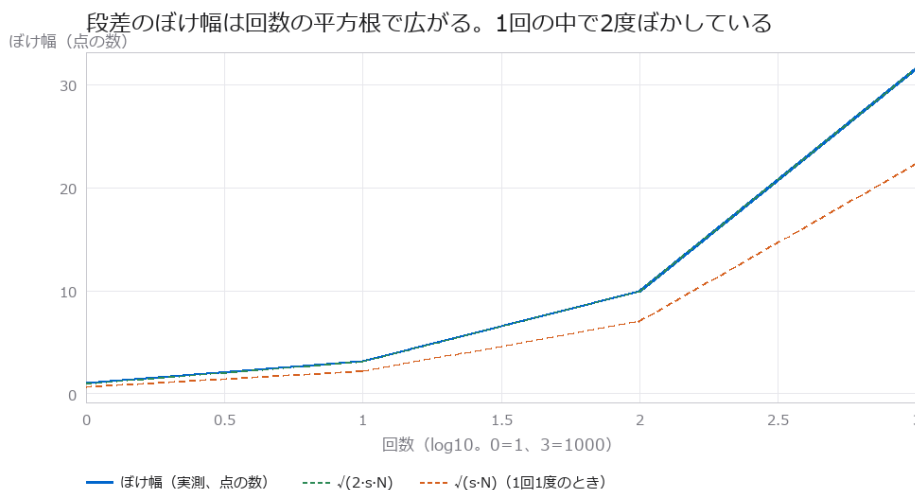
****既定のままだと、1点も変わらなかった****（100回かけて、変わった点 0 個）。Pin
Border（縁を止める）が入っていて、開いた線では全部の点が縁とみなされるため。線をぼかすときは
Pin Border を切る。

****切ると、1回のぼかしは「 $v \leftarrow v + s \cdot ((\text{左} + \text{右})/2 - v)$ 」を2度くり返したものと一致した****（Step
Size $s = 0.5$ 、1～1000回でずれ最大 0.0000021）。1度だけの計算とは最大 0.08 ずれる。Odd / Even
Step Size という2つのつまみがあることとも合う（奇数・偶数の段で1度ずつ）。

****段差のぼけ幅（標準偏差）は、回数の平方根で広がった。* 1・10・100・1000 回で
1・3.1623・10・31.6224 点ぶん（ $= \sqrt{(2 \cdot s \cdot N)}$ ）。ぼけ幅を2倍にしたいなら、回数を4倍にする。**

回数と、ぼけ方

ぼけ幅 = となりととの差を分布とみなしたときの標準偏差（点の数で）。端の点は、となり1点に近づく。



実測（青）は $\sqrt{(2 \cdot s \cdot N)}$ の線に重なる。

回数	2度ずつの計算とのずれ	1度ずつの計算とのずれ	ぼけ幅	$\sqrt{(2 \cdot s \cdot N)}$	秒
1	0.0000000	0.0625	1.0000	1.0000	0.0120
10	0.0000001	0.0832	3.1623	3.1623	0.0120
100	0.0000002	0.0831	10.0000	10.0000	0.0135
1000	0.0000021	0.0830	31.6224	31.6228	0.0233

わかったこと

- ・開いた線では Pin Border を切る。入れたままだと何も起きない（エラーも出ない）。
- ・1回 = 2度のぼかし。Step Size 0.5 なら、となりの平均との差の半分だけ近づく計算を2度。
- ・ぼけ幅は $\sqrt{\text{回数}}$ 。4倍の回数で2倍の幅。

次にやること

- ・面（格子）の上で、ぼけ幅が同じ式になるか
- ・Volume Preserving モードで、段差の高さがどう変わるか

copytopoints の向きは orient が N に勝つ — N は +z を、up は +y を合わせる。scale と pscale は掛け算

2026-09-24

何を作ったか

+z 方向の矢（原点・(0,0,1)・(0,1,0) の3点）を、属性を変えた点1つにコピーして、矢の z 軸と y 軸がどこを向いたか・どれだけ伸びたかを調べた。

****N だけなら、元の +z が N の向きになる**** (N = (1,0,0) で z 軸 → (1, 0, 0)) 。y 軸は (0, 1, 0) のまま。

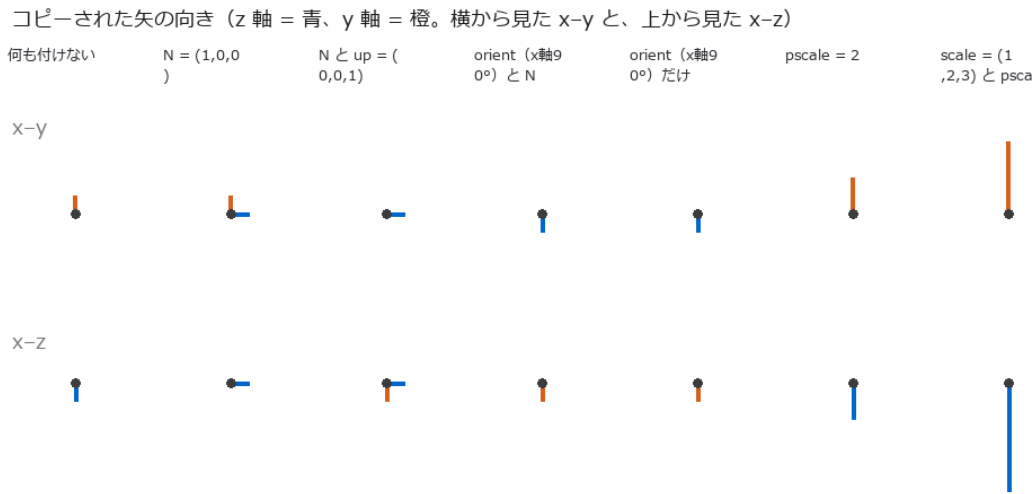
****up を足すと、元の +y が up の向きになる**** (up = (0,0,1) で y 軸 → (0, 0, 1)) 。N だけでは決まらない「ひねり」を up で決める。

****orient があると、N より orient が勝つ。**** N = (1,0,0) と、x 軸まわり 90° の orient を両方付けると、z 軸は (0, -1, 0) (orient だけのときと同じ) 。N の向き (1,0,0) は無視された。

****pscale は全体の倍率、scale は軸ごとの倍率で、両方あれば掛け算。**** pscale 2 で矢の長さ 2、scale (1,2,3) と pscale 2 で z 軸 6・y 軸 4 (= 3×2、2×2) 。

点の属性と、コピーの向き・大きさ

矢の z 軸 = 原点から (0,0,1) だった点へのベクトル。y 軸 = (0,1,0) だった点へのベクトル。



N は z 軸を、up は y 軸を動かす。orient があれば N は効かない。

点の属性	z 軸の行き先	y 軸の行き先	z の長さ	y の長さ
何も付けない	(0, 0, 1)	(0, 1, 0)	1	1
N = (1,0,0)	(1, 0, 0)	(0, 1, 0)	1	1
N と up = (0,0,1)	(1, 0, 0)	(0, 0, 1)	1	1
orient (x軸90°) と N	(0, -1, 0)	(0, 0, 1)	1	1
orient (x軸90°) だけ	(0, -1, 0)	(0, 0, 1)	1	1
pscale = 2	(0, 0, 2)	(0, 2, 0)	2	2

点の属性	z 軸の行き先	y 軸の行き先	z の長さ	y の長さ
scale = (1,2,3) と pscale 2	(0, 0, 6)	(0, 4, 0)	6	4

わかったこと

- ・ N は +z、up は +y を合わせる。コピーする形は、+z を「前」にして作っておく。
- ・ orient があれば N と up は使われない。向きがおかしいときは、上流で orient が残っていないかを見る。
- ・ pscale × scale。全体の大きさと、軸ごとの伸びを別々に付けられる。

次にやること

- ・ rot 属性や transform（行列）属性があるときの優先順位
- ・ v（速度）だけがあるとき、向きに使われるか

pointsfromvolume の点は、格子なら1点 $s^3 \cdot$ 四面体なら $s^3/\sqrt{2}$ — 箱では面の上にも並ぶ

2026-09-24

何を作ったか

箱（ $2 \times 1 \times 1$ ）と球（半径1）の中を、pointsfromvolume で間隔 s の点で埋め、点の数を数えた。

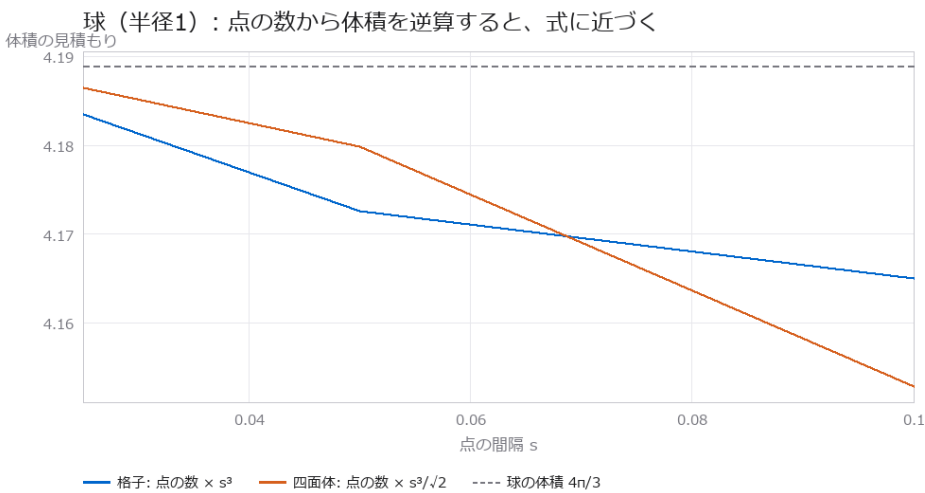
**格子（Grid）で埋めると、1点あたりの体積は s^3 。 ** 球では、点の数 $\times s^3$ が $s = 0.025$ で 4.1835（体積 4.1888、-0.13%）。

**四面体（Tetrahedral）で埋めると、同じ間隔で約1.41倍の点が入る。 ** 点の数 $\times s^3/\sqrt{2}$ が 4.1865（-0.05%）。面心立方の詰め方と同じ密度。

**箱を格子で埋めると、点は面の上にも並ぶ。 ** 点の数は $(2/s + 1) \cdot (1/s + 1)^2$ と3通りすべて一致（ $s = 0.1$ で 2,541 = $21 \times 11 \times 11$ ）。そのため、点の数 $\times s^3$ は箱の体積より多い（ $s = 0.1$ で 2.541、体積は 2）。点を粒とみなすと、粒の中心が面の上に来る。

形・並べ方・間隔と、点の数

球は polymesh 96 $\times$ 192。



間隔を細かくすると、格子（ s^3 ）も四面体（ $s^3/\sqrt{2}$ ）も球の体積に近づく。

形	並べ方	間隔	点の数	点 $\times s^3$	点 $\times s^3/\sqrt{2}$	体積	秒
box	grid	0.1	2,541	2.5410	1.7968	2.0000	0.222
box	grid	0.05	18,081	2.2601	1.5981	2.0000	0.164
box	grid	0.025	136,161	2.1275	1.5044	2.0000	0.107
box	tetrahedral	0.1	3,051	3.0510	2.1574	2.0000	0.177
box	tetrahedral	0.05	23,287	2.9109	2.0583	2.0000	0.132
box	tetrahedral	0.025	183,447	2.8664	2.0268	2.0000	0.225
sphere	grid	0.1	4,165	4.1650	2.9451	4.1888	0.106
sphere	grid	0.05	33,381	4.1726	2.9505	4.1888	0.197
sphere	grid	0.025	267,741	4.1835	2.9581	4.1888	0.119

形	並べ方	間隔	点の数	点×s ³	点×s ³ /√2	体積	秒
sphere	tetrahedral	0.1	5,873	5.8730	4.1528	4.1888	0.185
sphere	tetrahedral	0.05	47,289	5.9111	4.1798	4.1888	0.211
sphere	tetrahedral	0.025	378,921	5.9206	4.1865	4.1888	0.161

わかったこと

- ・ 格子は s³、四面体は s³/√2。同じ間隔なら四面体のほうが約1.41倍詰まる。
- ・ 箱では面の上にも点が並ぶ。粒のシミュレーションで体積をそろえたいときは注意。
- ・ 間隔を半分にすると点は8倍。0.025 で球の中に約27万～38万点。

次にやること

- ・ Jitter Scale を入れたときの点の数
- ・ SDF を渡したとき（Source = SDF）に数が変わるか

platonic の Radius

は、四面体・八面体・二十面体では頂点までの距離 — 立方体は辺が1、十二面体は0.9933

2026-09-24

何を作ったか

platonic で Radius = 1
にして5種類を作り、中心から頂点までの距離・辺・面積・体積を測った。外接球の半径が 1
のときの式と比べる。

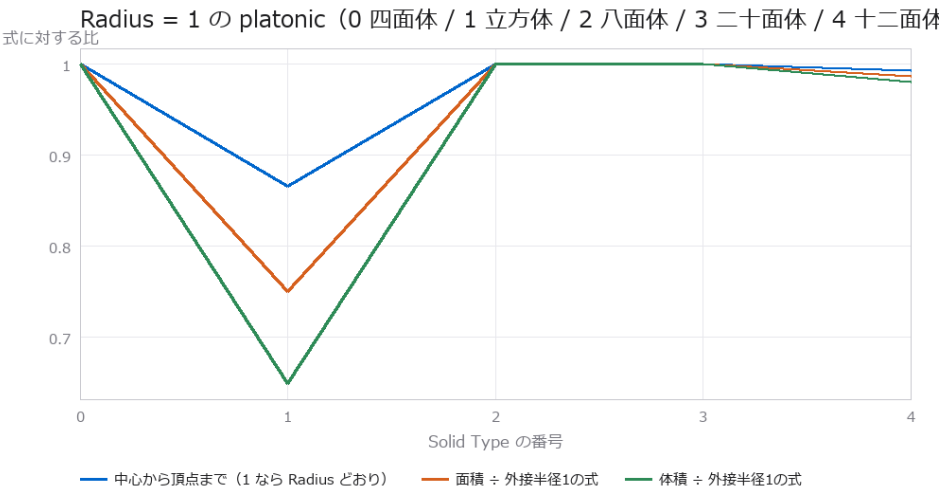
**四面体・八面体・二十面体は、Radius
がそのまま頂点までの距離。 **面積と体積も式と6桁一致した（二十面体の面積 9.574541、体積
2.536151）。

**立方体は違う。 **頂点までは $0.866025 (= \sqrt{3}/2)$ で、辺が 1。 Radius = 1 は「一辺 1
の立方体」になる。面積 6・体積 1。

**十二面体も1からずれる。 **頂点までは 0.993318、辺は 0.708876。 外接半径 1 の式（辺
0.713644）より -0.67%
小さい。形は正十二面体のまま（どの頂点も同じ距離）で、大きさだけが違う。なぜ 0.9933
なのかは確かめていない。

Radius = 1 の5種類

式は外接半径1のとき。 Solid Type の 5（Soccer Ball）と 6（Utah
Teapot）は正多面体ではないので外した。



青が1からずれるのは立方体（0.866）と十二面体（0.993）だけ。

形	点	面	頂点まで	辺	面積	式の面積	体積	式の体積	秒
Tetrahedron	4	4	1.00000 1	1.63299 5	4.61880 2	4.61880 2	0.51320 0	0.51320 0	0.0078
Cube	8	6	0.86602 5	1.00000 0	6.00000 0	8.00000 0	1.00000 0	1.53960 1	0.0050

形	点	面	頂点まで	辺	面積	式の面積	体積	式の体積	秒
Octahedron	6	8	1.00000 0	1.41421 4	6.92820 3	6.92820 3	1.33333 3	1.33333 3	0.0050
Icosahedron	12	20	1.00000 0	1.05146 3	9.57454 1	9.57454 1	2.53615 1	2.53615 1	0.0049
Dodecahedron	20	12	0.99331 8	0.70887 6	10.3745 77	10.5146 22	2.72970 6	2.78516 4	0.0049

わかったこと

- Radius は形によって意味が違う。四面体・八面体・二十面体は外接半径、立方体は一辺。
- 十二面体を大きさ合わせに使うなら、頂点までを測ってから。Radius=1 で 0.9933。
- どれも一瞬で作れる。1個 0.01 秒以下。

次にやること

- 十二面体の 0.9933 の出どころ
- Soccer Ball の面の数と面積

circle の Polygon は頂点が円周に乗る正 n 角形 — 弧の Divisions は辺の数

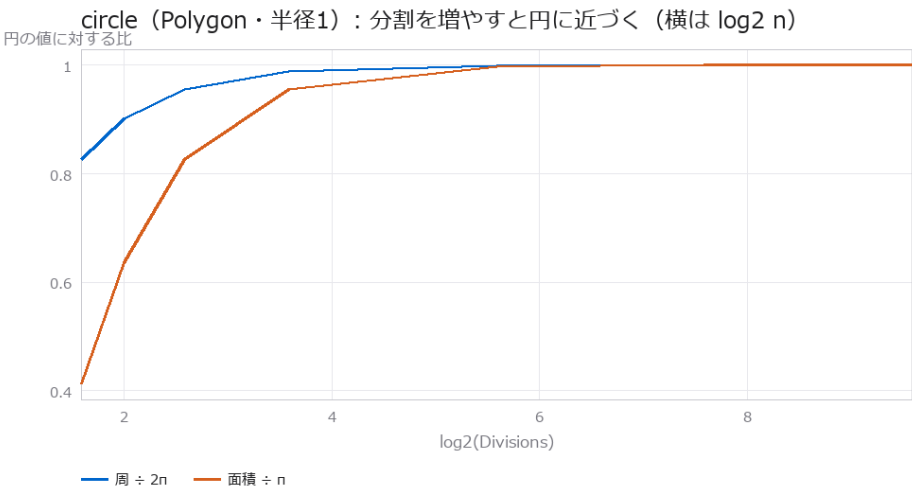
2026-09-24

何を作ったか

半径 1 の circle を Polygon にして、Divisions を 3～768 と変え、周と面積を測った。

頂点は全部ちょうど半径1の上。周は $2n \cdot \sin(n/n)$ 、面積は $(n/2) \cdot \sin(2n/n)$ と一致した（7通りで差は最大 $2e-06$ ）。
n = 12 で面積は 3（n より -4.5%）、n = 768 で 3.141556。円より少し小さく出るのは、辺が円の内側を通るから。
弧（0～90°）にすると、Divisions は辺の数になる。
Open Arc で Divisions = 4 なら点は 5、12 なら 13。Closed Arc は中心に1点足して閉じた扇形（周 3.560723 = 弧 + 半径2本）、Sliced Arc は中心から切った三角形が Divisions 枚（4 なら 4 枚）になる。

閉じた円: Divisions と周・面積



面積のほうが周より遅れて近づく。

Divisions	点	半径 (最小～最大)	周	式の周	面積	式の面積	秒
3	3	1～1	5.196152	5.196152	1.299038	1.299038	0.0001
4	4	1～1	5.656854	5.656854	2.000000	2.000000	0.0000
6	6	1～1	6.000000	6.000000	2.598076	2.598076	0.0000
12	12	1～1	6.211657	6.211657	3.000000	3.000000	0.0000
48	48	1～1	6.278700	6.278700	3.132628	3.132629	0.0000
192	192	1～1	6.282905	6.282905	3.141032	3.141032	0.0000
768	768	1～1	6.283168	6.283168	3.141556	3.141558	0.0001

弧 (Arc Angles 0～90) の3種類

Sliced Arc の面積は1枚目の三角形だけ。

Arc Type	Divisions	点	面	閉じている	周（1枚目）	面積（1枚目）
openarc	4	5	1	いいえ	1.560723	0.000000
openarc	12	13	1	いいえ	1.569675	0.000000
closedarc	4	6	1	はい	3.560723	0.765367
closedarc	12	14	1	はい	3.569675	0.783157
slicedarc	4	6	4	はい	2.390181	0.191342
slicedarc	12	14	12	はい	2.130806	0.065263

わかったこと

- 円の Polygon は内接多角形。面積は必ず n より小さい（12分割で -4.5% ）。
- 弧の Divisions は辺の数。点は Divisions + 1（Closed Arc はさらに中心の1点）。
- Sliced Arc はバラバラの三角形。ピザを切り分けた形で、面の数 = Divisions。

次にやること

- NURBS の円を convert したときの点の並び
- Rows/Columns が効く楕円（Radius を2つ別にする）

torus の Rows は管の断面、Columns は大きな輪 — 体積は2つの内接多角形の積で決まる

2026-09-24

何を作ったか

Radius = (1, 0.25) の torus (Polygon) で、Rows と Columns を片方ずつ 6~96 に変え、面積と体積を測った。式は面積 $4\pi^2 Rr = 9.869604$ 、体積 $2\pi^2 Rr^2 = 1.233701$ 。

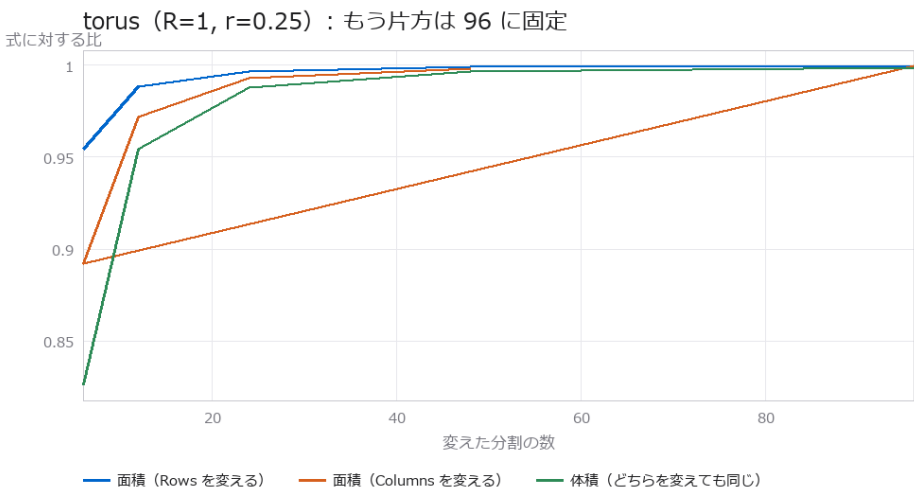
****Rows** は管の断面を刻み、**Columns** は大きな輪を刻む。****Rows** を増やすと点の高さの種類が増え (Rows 6 で 3、96 で 51)、**Columns** では変わらない。

****体積**は Rows と Columns を入れ替えても同じ値になった****** (Rows 6 でも Columns 6 でも 1.019534)。断面も輪も内接正多角形なので、体積 = 式 $\times f(\text{Rows}) \times f(\text{Columns})$ 、 $f(n) = n \cdot \sin(2\pi/n)/2\pi$ と置くと、9通りすべて $4e-07$ 以内で合った。

****面積**は入れ替えると違う。****6分割**のとき、Rows を減らすと 9.4206 (-4.5%)、Columns を減らすと 9.8634 (-0.1%)。大きな輪 (Columns) を粗くするほうが面積は大きく減る。

Rows・Columns と面積・体積

予測の体積 = $2\pi^2 Rr^2 \times f(\text{Rows}) \times f(\text{Columns})$ 。



緑の体積は2本とも同じ線に重なる。

Rows	Columns	点	高さの種類	面積	面積の比	体積	予測の体積	秒
6	96	576	3	9.420574	0.9545	1.019534	1.019534	0.0007
12	96	1,152	7	9.752896	0.9882	1.177256	1.177256	0.0002
24	96	2,304	13	9.837052	0.9967	1.218785	1.218785	0.0003
48	96	4,608	26	9.858160	0.9988	1.229302	1.229302	0.0007
96	96	9,216	51	9.863441	0.9994	1.231940	1.231940	0.0012
96	6	576	51	8.803202	0.8920	1.019534	1.019534	0.0003
96	12	1,152	51	9.590021	0.9717	1.177256	1.177256	0.0002
96	24	2,304	51	9.797642	0.9927	1.218785	1.218785	0.0003

Rows	Columns	点	高さの種類	面積	面積の比	体積	予測の体積	秒
96	48	4,608	51	9.850246	0.9980	1.229302	1.229302	0.0005

わかったこと

- Rows = 管の太さ方向、Columns = 輪の周方向。名前から逆に思いやすいので注意。
- 体積は2つの分割の掛け算。どちらを粗くしても同じだけ減る。
- 面積は輪の分割に敏感。見た目を保ちたいなら Columns を先に確保する。

次にやること

- Radius の r を R に近づけたとき（自分に食い込む手前）
- polysoup で作ったときの速さ

mirror の継ぎ目は「元と鏡像の距離」が Consolidate Seam 未満ならまとまる ー 境目ちょうどはまとまらない

2026-09-24

何を作ったか

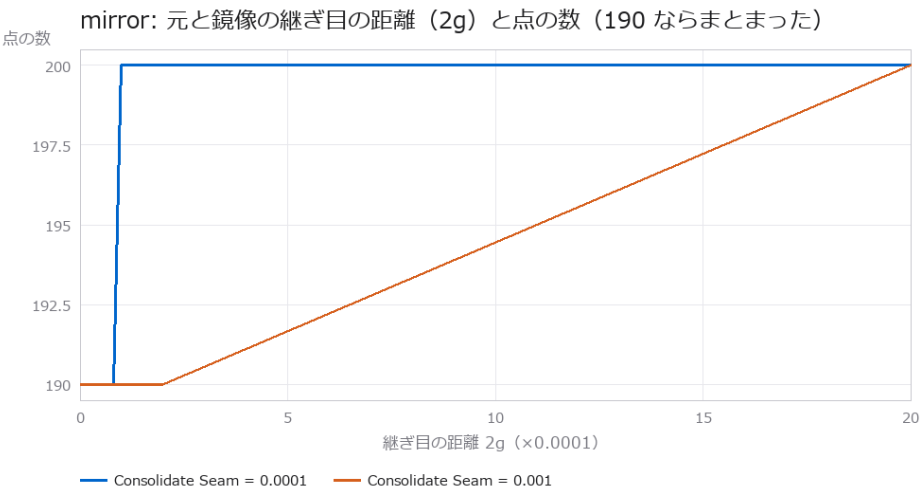
10×10 点の grid を、縁が面から g だけ離れるように置いて mirror し、点の数を数えた。全部まとまれば 190、まとまらなければ 200。

**比べられるのは、面からの距離 g ではなく、元と鏡像の距離 $2g$ 。 **Consolidate Seam が既定の 0.0001 のとき、 $g = 0.00004$ (距離 0.00008) はまとまり、 $g = 0.00005$ (距離 0.0001、ちょうど境目) はまとまらなかった。

**まとまった点は面の上 ($x = 0$) に寄る。 **元の縁が 0.00004 離れていても、出力の継ぎ目は $x = 0$ の1列だけ。まとまらないと $\pm g$ の2列が残り、見た目はつながっていても面は割れている。

**Enable Consolidate Seam を切ると、隙間 0 でも 200 点。 **同じ位置に点が2つずつ重なる。面の数はどの場合も 162 (81×2) で変わらない。

隙間・ Consolidate Seam ・点の数



距離がつまみの値に届いたところで 200 に跳ねる。

面からの隙間 g	継ぎ目の距離 $2g$	Consolidate Seam	有効	点	面	継ぎ目の x	秒
0	0	0.0001	入	190	162	0	0.0005
0	0	0.0001	切	200	162	0	0.0002
0	0	0.001	入	190	162	0	0.0002
0	0	0.001	切	200	162	0	0.0002
2e-05	4e-05	0.0001	入	190	162	0	0.0002
2e-05	4e-05	0.0001	切	200	162	-2e-05 / 2e-05	0.0002
2e-05	4e-05	0.001	入	190	162	0	0.0002

面からの隙間 g	継ぎ目の距離 2g	Consolidate Seam	有効	点	面	継ぎ目の x	秒
2e-05	4e-05	0.001	切	200	162	-2e-05 / 2e-05	0.0002
4e-05	8e-05	0.0001	入	190	162	0	0.0002
4e-05	8e-05	0.0001	切	200	162	-4e-05 / 4e-05	0.0002
4e-05	8e-05	0.001	入	190	162	0	0.0002
4e-05	8e-05	0.001	切	200	162	-4e-05 / 4e-05	0.0002
5e-05	0.0001	0.0001	入	200	162	-5e-05 / 5e-05	0.0002
5e-05	0.0001	0.0001	切	200	162	-5e-05 / 5e-05	0.0002
5e-05	0.0001	0.001	入	190	162	0	0.0002
5e-05	0.0001	0.001	切	200	162	-5e-05 / 5e-05	0.0002
6e-05	0.00012	0.0001	入	200	162	-6e-05 / 6e-05	0.0002
6e-05	0.00012	0.0001	切	200	162	-6e-05 / 6e-05	0.0002
6e-05	0.00012	0.001	入	190	162	0	0.0002
6e-05	0.00012	0.001	切	200	162	-6e-05 / 6e-05	0.0002
0.0001	0.0002	0.0001	入	200	162	-0.0001 / 0.0001	0.0002
0.0001	0.0002	0.0001	切	200	162	-0.0001 / 0.0001	0.0002
0.0001	0.0002	0.001	入	190	162	0	0.0002
0.0001	0.0002	0.001	切	200	162	-0.0001 / 0.0001	0.0002
0.001	0.002	0.0001	入	200	162	-0.001 / 0.001	0.0002
0.001	0.002	0.0001	切	200	162	-0.001 / 0.001	0.0002
0.001	0.002	0.001	入	200	162	-0.001 / 0.001	0.0002
0.001	0.002	0.001	切	200	162	-0.001 / 0.001	0.0002

わかったこと

- ・面から g 離れた縁は、 $g < \text{Consolidate Seam} \div 2$ でまとまる。比べるのは元と鏡像の距離。
- ・境目ちょうどはまとまらない（「未満」の比較）。
- ・半分だけ作って mirror するなら、縁を面の上に置く。ずれがあってもまとまった点は面へ寄る。

次にやること

- Only Consolidate Unshared Edges を切ったとき
- Distance を入れて面をずらしたとき

clip で切った球の面積は $2\pi h$ — 比は切る高さに関係なく、球全体と同じだけずれる

2026-09-24

何を作ったか

半径 1 の球 (Polygon Mesh) を、Direction = (0,1,0)、Distance = c の面で clip し、残った面積を測った。球帯の面積は高さ h だけで決まる ($2\pi Rh$)。

**Above は面より上 ($y \geq c$)、Below は下が残る。 **Distance は原点からその向きに進んだ距離。

**面積は $2\pi h$ に合う。 **分割 192×384 で、8通りすべて式から 0.014% 以内。式に対する比は、どの高さでもほぼ球全体の比 (0.99994) と同じ。切り口の位置によらず、同じ分割の網を切っているだけだから。

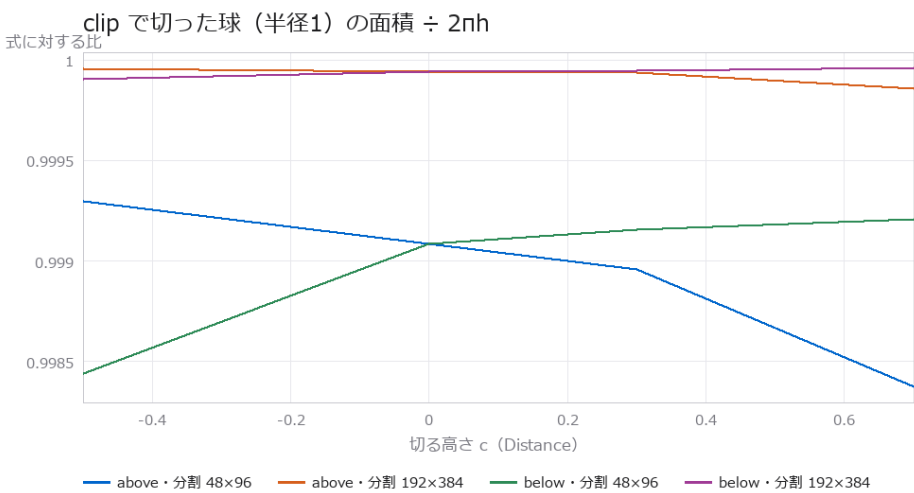
**切り口の点は面の上に乗る。 **どの場合も列の数 (48 分割なら 96、192 分割なら 384) だけ、 $y = c$ ちょうどの点ができた。

**Both は2つに分けるが、切り口の点は共有のまま。 **48 分割・ $c = 0.3$ で、元の球 4,418 点・4,512 面が 4,514 点 (+96) ・4,608

面 (+96)。点は複製されないので、上下を引き離すには別に割る処理が要る。

切る高さ・残す側・面積

式: Above は $2\pi(1 - c)$ 、Below は $2\pi(1 + c)$ 。Both は球全体。



線が水平 = 比が高さによらない。細かい分割 (192) ほど 1 に近い。

分割	c	Keep	点	面	y の範囲	面上の点	面積	式	秒
48×96	-0.5	above	3,073	3,072	-0.5~1	96	9.41817 1	9.42477 8	0.0023
48×96	-0.5	below	1,537	1,536	-1~-0.5	96	3.13669 9	3.14159 3	0.0015
48×96	-0.5	both	4,514	4,608	-1~1	96	12.5548 70	—	0.0008
48×96	0	above	2,305	2,304	0~1	96	6.27743 5	6.28318 5	0.0013

分割	c	Keep	点	面	y の範囲	面上の点	面積	式	秒
48×96	0	below	2,305	2,304	-1～0	96	6.27743 5	6.28318 5	0.0019
48×96	0	both	4,514	4,608	-1～1	96	12.5548 70	—	0.0007
48×96	0.3	above	1,825	1,824	0.3～1	96	4.39364 3	4.39823 0	0.0016
48×96	0.3	below	2,785	2,784	-1～0.3	96	8.16122 7	8.16814 1	0.0014
48×96	0.3	both	4,514	4,608	-1～1	96	12.5548 70	—	0.0008
48×96	0.7	above	1,153	1,152	0.7～1	96	1.88189 5	1.88495 6	0.0014
48×96	0.7	below	3,457	3,456	-1～0.7	96	10.6729 75	10.6814 15	0.0015
48×96	0.7	both	4,514	4,608	-1～1	96	12.5548 70	—	0.0008
192×384	-0.5	above	49,153	49,152	-0.5～1	384	9.42437 0	9.42477 8	0.0112
192×384	-0.5	below	24,577	24,576	-1～-0.5	384	3.14129 5	3.14159 3	0.0152
192×384	-0.5	both	73,346	73,728	-1～1	384	12.5656 65	—	0.0053
192×384	0	above	36,865	36,864	0～1	384	6.28283 3	6.28318 5	0.0118
192×384	0	below	36,865	36,864	-1～0	384	6.28283 3	6.28318 5	0.0146
192×384	0	both	73,346	73,728	-1～1	384	12.5656 65	—	0.0063
192×384	0.3	above	29,569	29,568	0.3～1	384	4.39795 7	4.39823 0	0.0135
192×384	0.3	below	44,161	44,160	-1～0.3	384	8.16770 8	8.16814 1	0.0139
192×384	0.3	both	73,346	73,728	-1～1	384	12.5656 65	—	0.0065
192×384	0.7	above	18,817	18,816	0.7～1	384	1.88468 9	1.88495 6	0.0155
192×384	0.7	below	54,913	54,912	-1～0.7	384	10.6809 77	10.6814 15	0.0103
192×384	0.7	both	73,346	73,728	-1～1	384	12.5656 65	—	0.0065

わかったこと

- Distance は原点から Direction の向きに進んだ位置。Above はその先（向きの側）を残す。
- 切り口は面の上にちょうど乗る。あとで cap や polyfill で塞ぐとき、平らに閉じる。

- ・ Both の上下は点を共有している。別々に動かすなら、切ったあとに割る処理を足す。

次にやること

- ・ 切り口を polyfill で塞いだときの体積（球冠の式）
- ・ Direction を斜めにしたとき

twist SOP の6つの操作 — 体積を保つのは Shear だけ。Squash は長さ $(1+s)$ 倍・太さ $1/(1+s)$ 倍

2026-09-24

何を作ったか

x が 0~1 (または 0~2) の箱に、Primary Axis = X で6つの Operation をかけ、体積と端の点の動きを測った。

**Shear はぴったり体積を保つ。 ** y が Strength $\times x$ だけずれ、囲む箱の高さは $1 + s \times \text{長さ}$ (長さ $2 \cdot s = 1$ で 3) 。

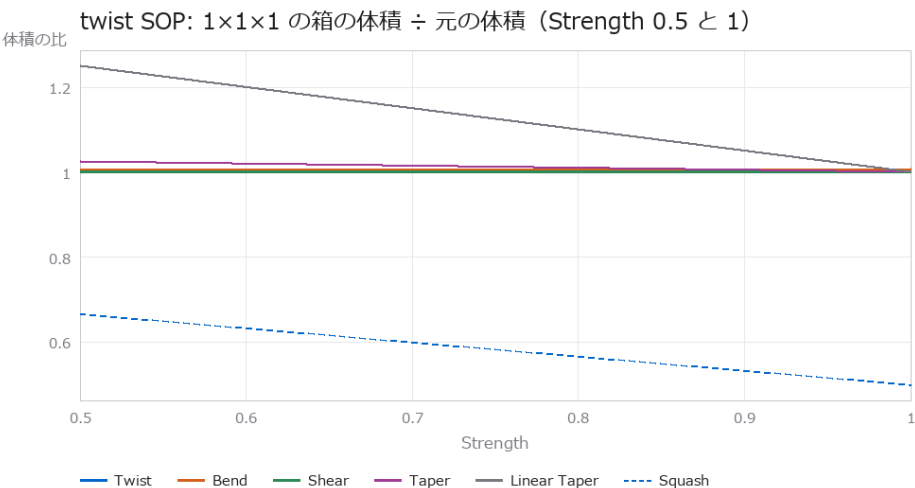
**Twist の Strength は、 x が 1 進むごとの回転角 (度) 。 ** $x = 1$ の角の点が $s = 2$ で 2° 回った。体積は 1.000641 倍と、わずかに増えた。側面の四角形がねじれて平らでなくなるため。

**Squash は長さを $(1 + s)$ 倍、太さを $1/(1 + s)$ 倍にする。 **体積は $1/(1 + s)$ 倍 ($s = 0.5$ で 0.666667、 $s = 1$ で 0.5) 。名前と違い、体積は保たれない。

**Taper と Linear Taper は、Secondary Axis (Y) の位置で x 方向の長さを変える。Strength = 1 で変化なし。 **Linear Taper は $y = -0.5$ で s 倍、 $+0.5$ で $1/s$ 倍、その間は直線 ($s = 0.5$ で 0.5~2 倍、体積 1.25 倍) 。Taper は $s^{(-y)}$ 倍 ($y = +0.5$ で 1.4142、 -0.5 で 0.7071) 。ただし Taper の体積 1.024674 倍は、この倍率を y で平均した値 1.020139 と 0.44% 合わない。理由は確かめていない。

**Bend の Strength の単位は確かめていない。 ** $s = 2$ で端の点が 0.05 ほど動くだけで、弧の角度として読めなかった。

操作・Strength・体積



Shear と Twist はほぼ 1。Squash は下がり、Taper 系は $s = 1$ で 1 に戻る。

長さ	Operation	Strength	元の体積	体積	比	囲む箱	秒
1	Twist	0.5	1	1.000322	1.000322	$1 \times 1.0087 \times 1.0087$	0.0001
1	Twist	1	1	1.000641	1.000641	$1 \times 1.0173 \times 1.0173$	0.0000

長さ	Operation	Strength	元の体積	体積	比	囲む箱	秒
1	Bend	0.5	1	1.004361	1.004361	$1.0087 \times 1.013 \times 1$	0.0001
1	Bend	1	1	1.008718	1.008718	$1.0174 \times 1.026 \times 1$	0.0000
1	Shear	0.5	1	1.000000	1.000000	$1 \times 1.5 \times 1$	0.0000
1	Shear	1	1	1.000000	1.000000	$1 \times 2 \times 1$	0.0000
1	Taper	0.5	1	1.024674	1.024674	$1.4142 \times 1 \times 1$	0.0000
1	Taper	1	1	1.000000	1.000000	$1 \times 1 \times 1$	0.0000
1	Linear Taper	0.5	1	1.250000	1.250000	$2 \times 1 \times 1$	0.0000
1	Linear Taper	1	1	1.000000	1.000000	$1 \times 1 \times 1$	0.0000
1	Squash	0.5	1	0.666667	0.666667	$1.5 \times 0.6667 \times 0.6667$	0.0000
1	Squash	1	1	0.500000	0.500000	$2 \times 0.5 \times 0.5$	0.0000
2	Twist	0.5	2	2.001282	1.000641	$2 \times 1.0173 \times 1.0173$	0.0000
2	Twist	1	2	2.002541	1.001270	$2 \times 1.0343 \times 1.0343$	0.0000
2	Bend	0.5	2	2.017449	1.008724	$2.0087 \times 1.0261 \times 1$	0.0000
2	Bend	1	2	2.034890	1.017445	$2.0174 \times 1.0522 \times 1$	0.0000
2	Shear	0.5	2	2.000000	1.000000	$2 \times 2 \times 1$	0.0000
2	Shear	1	2	2.000000	1.000000	$2 \times 3 \times 1$	0.0000
2	Taper	0.5	2	2.049347	1.024674	$2.8284 \times 1 \times 1$	0.0000
2	Taper	1	2	2.000000	1.000000	$2 \times 1 \times 1$	0.0000
2	Linear Taper	0.5	2	2.500000	1.250000	$4 \times 1 \times 1$	0.0000
2	Linear Taper	1	2	2.000000	1.000000	$2 \times 1 \times 1$	0.0000
2	Squash	0.5	2	1.333333	0.666667	$3 \times 0.6667 \times 0.6667$	0.0000
2	Squash	1	2	1.000000	0.500000	$4 \times 0.5 \times 0.5$	0.0000

$x = 1 \cdot z = 0.5$ の端の点の行き先（長さ 1 の箱）

Operation	Strength	元の y	行き先 (x, y, z)
Twist	0.5	-0.5	1, -0.4956, 0.5043

Operation	Strength	元の y	行き先 (x, y, z)
Twist	0.5	0.5	1, 0.5043, 0.4956
Twist	2	-0.5	1, -0.4822, 0.5171
Twist	2	0.5	1, 0.5171, 0.4822
Bend	0.5	-0.5	1.0022, -0.4956, 0.5
Bend	0.5	0.5	0.9934, 0.513, 0.5
Bend	2	-0.5	1.0086, -0.4825, 0.5
Bend	2	0.5	0.9725, 0.5517, 0.5
Shear	0.5	-0.5	1, 0, 0.5
Shear	0.5	0.5	1, 1, 0.5
Shear	2	-0.5	1, 1.5, 0.5
Shear	2	0.5	1, 2.5, 0.5
Taper	0.5	-0.5	0.7071, -0.5, 0.5
Taper	0.5	0.5	1.4142, 0.5, 0.5
Taper	2	-0.5	1.4142, -0.5, 0.5
Taper	2	0.5	0.7071, 0.5, 0.5
Linear Taper	0.5	-0.5	0.5, -0.5, 0.5
Linear Taper	0.5	0.5	2, 0.5, 0.5
Linear Taper	2	-0.5	2, -0.5, 0.5
Linear Taper	2	0.5	0.5, 0.5, 0.5
Squash	0.5	-0.5	1.5, -0.3333, 0.3333
Squash	0.5	0.5	1.5, 0.3333, 0.3333
Squash	2	-0.5	3, -0.1667, 0.1667
Squash	2	0.5	3, 0.1667, 0.1667

わかったこと

- ・体積を保ちたいなら Shear。Twist もほぼ保つ（ねじれた面の分だけずれる）。
- ・Squash は体積を保たない。伸ばして細くする量が、同じ $(1+s)$ で釣り合っていない。
- ・Taper 系は Strength = 1 が「何もしない」。0 にすると片側がつぶれる。

次にやること

- ・Taper の体積が平均と合わない理由
- ・Bend の Strength と曲がる角度の関係

tube の円錐台は式 × 内接多角形の比 — Radius の1つ目が上、円錐の先は点が重なったまま

2026-09-24

何を作ったか

tube (Polygon、End Caps 入り、Height 1) で、Radius を (1, 0) ・ (1, 0.5) ・ (0.5, 1) にし、Columns を 12~384 と変えて体積と面積を測った。

Radius の1つ目が上 (y = +0.5) 、2つ目が下。(0.5, 1) では上の半径が 0.5 になった。上下を入れ替えても、体積と面積は同じ値。

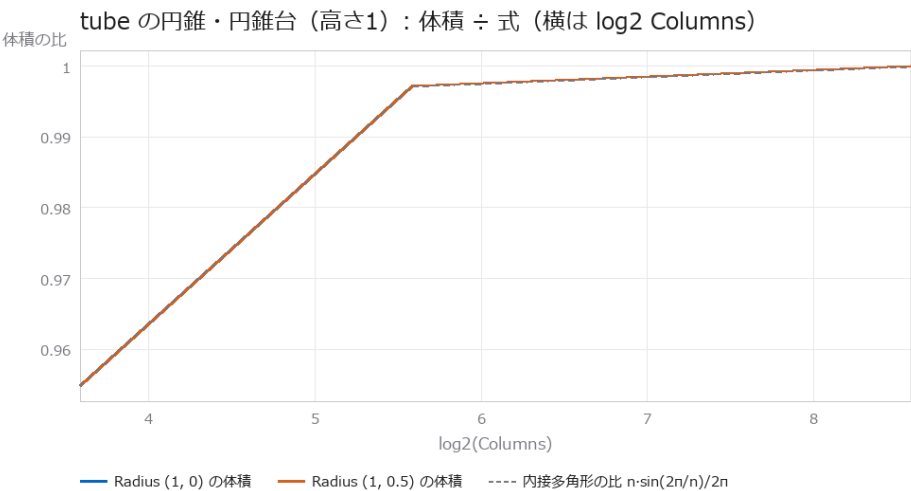
体積は、円錐台の式 $\times n \cdot \sin(2\pi/n)/2\pi$ に合う (9通りで差は最大 1e-05) 。断面が内接正 n 角形なので、円 (実験143) と同じだけ小さくなる。Columns 12 の円錐は 1.00001 (式 1.047198) 。

円錐 (Radius の片方 0) でも、先の点は Columns の数だけ重なって残る。Columns 12 で点は 24 (上の輪 12 + 先 12) 。先を1点にしたいときは fuse 要る。

面積も式 (側面 + 蓋2枚) に近づく。Columns 384 で、円錐 7.584235 (式 7.584476) 。

Radius ・ Columns と体積 ・ 面積

予測の体積 = $nh(r_1^2 + r_1r_2 + r_2^2)/3 \times n \cdot \sin(2\pi/n)/2\pi$ 。式の面積 = 側面 $n(r_1 + r_2)\sqrt{((r_1 - r_2)^2 + h^2)}$ + 蓋 $n(r_1^2 + r_2^2)$ 。



円錐も円錐台も、点線 (内接多角形の比) に重なる。

Radius	Column s	点	面	上の半径	体積	予測	式の体積	面積	式の面積	秒
(1, 0)	12	24	14	1	1.00001 0	1.00000 0	1.04719 8	7.31814 3	7.58447 6	0.0002
(1, 0)	48	96	50	1	1.04422 0	1.04421 0	1.04719 8	7.56761 2	7.58447 6	0.0001
(1, 0)	384	768	386	1	1.04716 1	1.04715 1	1.04719 8	7.58423 5	7.58447 6	0.0003
(1, 0.5)	12	24	14	1	1.75000 0	1.75000 0	1.83259 6	8.92362 4	9.19560 2	0.0001

Radius	Column s	点	面	上の半径	体積	予測	式の体積	面積	式の面積	秒
(1, 0.5)	48	96	50	1	1.82736 7	1.82736 7	1.83259 6	9.17838 3	9.19560 2	0.0001
(1, 0.5)	384	768	386	1	1.83251 4	1.83251 4	1.83259 6	9.19533 4	9.19560 2	0.0002
(0.5, 1)	12	24	14	0.5	1.75000 0	1.75000 0	1.83259 6	8.92362 4	9.19560 2	0.0001
(0.5, 1)	48	96	50	0.5	1.82736 7	1.82736 7	1.83259 6	9.17838 3	9.19560 2	0.0001
(0.5, 1)	384	768	386	0.5	1.83251 4	1.83251 4	1.83259 6	9.19533 4	9.19560 2	0.0002

わかったこと

- Radius の1つ目 = 上、2つ目 = 下。
- tube の円錐は先が閉じていない。点が Columns 個重なっているなので、fuse してから使う。
- 体積の不足は Columns だけで決まる。12 で -4.5%、48 で -0.3%、384 で -0.004%。

次にやること

- Rows を増やしたときに体積が変わらないこと
- fuse で先を1点にしたあとの法線

divide の Bricker は形の端 (+ Offset) から Size おきに線を引く — Convex は n 角形を n-2 枚の三角形か、その半分の四角形に

2026-09-24

何を作ったか

1×1 の正方形1枚（中心が原点）を Bricker Polygons で、正 n 角形を Convex Polygons で割り、枚数を数えた。

**Bricker は、形の囲む箱の端 (+ Offset) から Size おきに切る線を引く。 **枚数は「正方形の内側を通る線の数 + 1」の2乗で、8通りすべて一致。 Size 0.3 だと線は $-0.2 \cdot 0.1 \cdot 0.4$ の3本で 16 枚（右端の升だけ 0.1 幅）。 Offset 0.05 だと $-0.45 \cdot -0.15 \cdot 0.15 \cdot 0.45$ の4本で 25 枚。 Size 0.5 は線が 0 の1本だけで 4 枚、 Offset 0.05 だと $-0.45 \cdot 0.05$ の2本で 9 枚。

**Convex で Maximum Edges = 3 なら $n - 2$ 枚の三角形、4 なら $(n - 2)/2$ 枚（切り上げ）。 **8通りすべてこの数。 奇数角形を 4 で割ると、三角形が1枚だけ混ざる（13角形で三角形1+四角形5）。

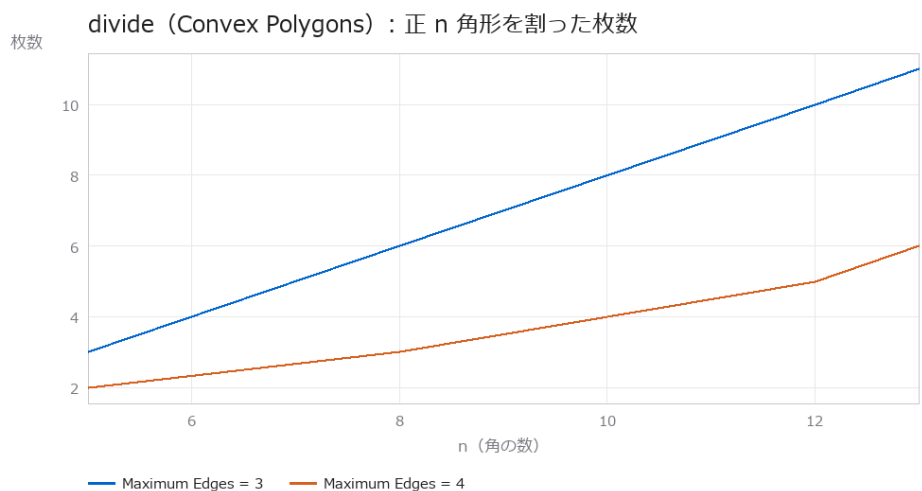
**どちらも面積は変わらない。 **割る前と後で6桁一致した。

Bricker: Size · Offset と枚数 (1×1 の正方形)

Offset は $x \cdot y \cdot z$ に同じ値。線は $-0.5 + \text{Offset} + k \cdot \text{Size}$ 。 予測 = (内側を通る線の数 + 1)²。

Size	Offset	面	予測	点	何角形か	面積	秒
0.5	0	4	4	9	4	1	0.0003
0.5	0.05	9	9	16	4	1	0.0001
0.25	0	16	16	25	4	1	0.0001
0.25	0.05	25	25	36	4	1	0.0001
0.3	0	16	16	25	4	1	0.0001
0.3	0.05	25	25	36	4	1	0.0001
0.1	0	100	100	121	4	1	0.0006
0.1	0.05	121	121	144	4	1	0.0005

Convex: 正 n 角形を割る



3 で割ると $n - 2$ 枚、4 で割るとその約半分。

n	Maximum Edges	面	予測	面ごとの角の数	面積	割る前
5	3	3	3	3・3・3	2.377641	2.377641
5	4	2	2	3・4	2.377641	2.377641
8	3	6	6	3・3・3・3・3・3	2.828427	2.828427
8	4	3	3	4・4・4	2.828427	2.828427
12	3	10	10	3・3・3・3・3・3・3・3・3・3	3.000000	3.000000
12	4	5	5	4・4・4・4・4	3.000000	3.000000
13	3	11	11	3・3・3・3・3・3・3・3・3・3・3	3.020701	3.020701
13	4	6	6	3・4・4・4・4・4	3.020701	3.020701

わかったこと

- Bricker の升目は形の端から並ぶ。Offset はその始まりをずらす量（Offset 0 でも端に線は増えない）。
- 割り切れない Size だと、反対側の端に細い升が残る。1 ÷ 0.3 なら 0.1 幅の列。
- Maximum Edges = 4 は、奇数角形だと三角形が1枚混ざる。全部四角にはならない。

次にやること

- Bricker を立体（box）にかけたときの枚数
- Compute Dual が出る面の数

sphere の Polygon は点 $10f^2+2$ ・面 $20f^2$ の測地球 — 同じ点の数なら Polygon Mesh より体積の不足が約3割小さい

2026-09-24

何を作ったか

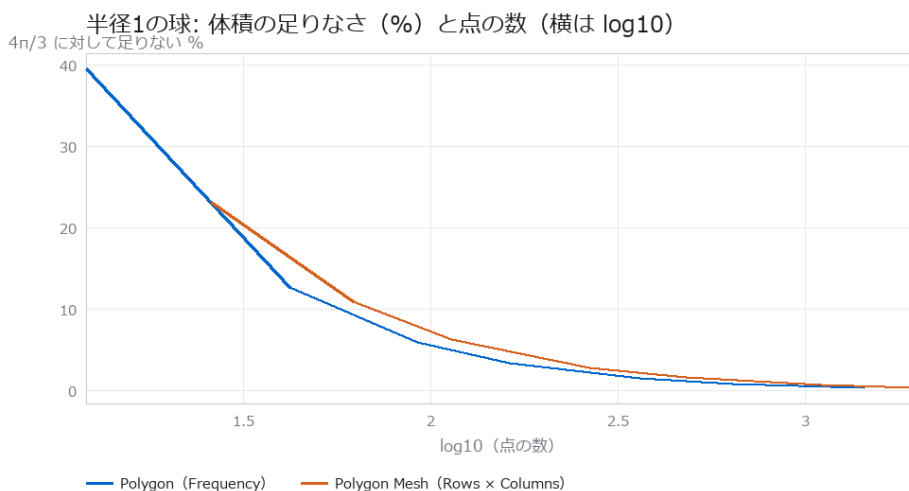
半径 1 の sphere を Primitive Type = Polygon にして Frequency f を 1~12 と変え、同じくらいの点の数の Polygon Mesh (Rows = Columns/2 + 1) と比べた。

**Polygon は二十面体を分けた測地球。 **点は $10f^2+2$ 、面は $20f^2$ (すべて三角形) で、7通りすべて一致。 $f = 1$ は正二十面体そのもの (面積 9.574541 ・体積 2.536151 は実験142 の二十面体と同じ値) 。頂点はどれも半径 1 ちょうどの上にある。

**同じ点の数なら、Polygon のほうが丸い。 **体積の不足 (%) × 点の数は、点が 160 を超えたあたりから Polygon で 約 548.2、Polygon Mesh で 約 775.9。どちらも「点の数に反比例して近づく」が、係数は Polygon のほうが 29% 小さい。 $f = 12$ (1,442 点) で不足 0.381%、Mesh 48 列 (1,106 点) で 0.712%。

**Polygon Mesh は極に三角形が集まる。 **面は四角形と三角形が混ざる。 Polygon は三角形だけで、大きさがそろっている。

Polygon (Frequency)



青 (Polygon) が、同じ点の数でいつも下 (不足が小さい) 。

Frequency	点	$10f^2+2$	面	$20f^2$	面の形	面積	体積	体積の不足 %	不足 × 点	秒
1	12	12	20	20	3角形	9.574541	2.536151	39.454	473.4	0.0002
2	42	42	80	80	3角形	11.665931	3.658712	12.655	531.5	0.0001
3	92	92	180	180	3角形	12.152844	3.942285	5.885	541.4	0.0001

Frequency	点	$10f^2+2$	面	$20f^2$	面の形	面積	体積	体積の不足 %	不足 × 点	秒
4	162	162	320	320	3角形	12.330941	4.047813	3.366	545.2	0.0001
6	362	362	720	720	3角形	12.460796	4.125362	1.514	548.2	0.0003
8	642	642	1,280	1,280	3角形	12.506794	4.152956	0.855	549.2	0.0002
12	1,442	1,442	2,880	2,880	3角形	12.539831	4.172813	0.381	550.0	0.0003

Polygon Mesh (比べる相手)

Rows × Columns	点	面	面の形	面積	体積	体積の不足 %	不足 × 点	秒
5 × 8	26	32	3・4角形	11.013439	3.218951	23.153	602.0	0.0002
7 × 12	62	72	3・4角形	11.861288	3.732051	10.904	676.0	0.0001
9 × 16	114	128	3・4角形	12.166689	3.926596	6.259	713.6	0.0001
13 × 24	266	288	3・4角形	12.387741	4.070552	2.823	750.8	0.0001
17 × 32	482	512	3・4角形	12.465694	4.121942	1.596	769.2	0.0002
25 × 48	1,106	1,152	3・4角形	12.521563	4.158971	0.712	787.3	0.0005
33 × 64	1,986	2,048	3・4角形	12.541153	4.171996	0.401	796.3	0.0008

わかったこと

- ・ Frequency f で点は $10f^2+2$ 。 f を倍にすると点も面も約4倍。
- ・ 少ない点で丸く見せたいなら Polygon。体積の不足が同じ点の数で約3割小さい。
- ・ UV や行列で扱いたいなら Polygon Mesh。 Rows と Columns の格子になっている。

次にやること

- ・ Polygon の球を UV 展開したときのゆがみ
- ・ remesh にかけたときに点の数がどれだけ変わるか

polybevel の Round は、分割を増やすと「角を丸めた箱」の体積に近づく — 平らな面取りは $1 - 6d^2 + (16/3)d^3$

2026-09-24

何を作ったか

1×1×1 の箱の辺を全部 polybevel (Offset = d) にかかけ、Fillet Shape と Divisions を変えて体積を測った。比べる式は、半径 d で角を丸めた箱 $(1-2d)^3 + 6d(1-2d)^2 + 3nd^2(1-2d) + (4/3)nd^3$ 。

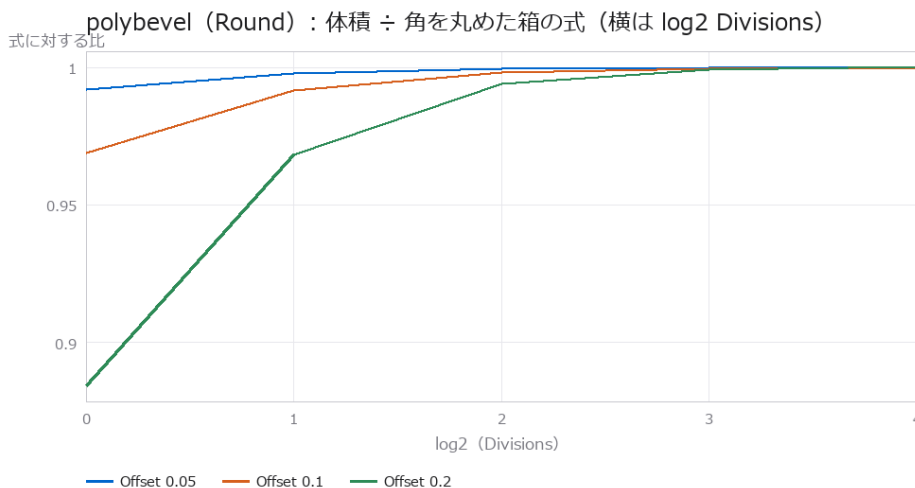
**Divisions = 1 の Round は Solid と同じ形。 **体積が3通りとも一致した。平らな面取りの体積は $1 - 6d^2 + (16/3)d^3$ と合った (差は最大 0e+00)。辺を削る 12 本ぶん ($6d^2$) から、角で2重に削った分 ($(16/3)d^3$) が戻る形。

**Round は、Divisions を倍にするたびに式との差が 1/3.6~1/6 に縮んだ (およそ 1/4)。 **Offset 0.1・Divisions 16 で 0.975521 (式 0.975587)。Offset は丸みの半径そのものとして効いている。

Offset 0.2・Divisions 16 だけは、式をわずかに超えた (0.907782 > 0.907705、+0.0085%)。角の丸みがちょうどの球面になっていない可能性があるが、確かめていない。

点の数は Divisions とともに増える (1 で 24、2 で 56、4 で 152、8 で 488、16 で 1,736)。Offset には左右されない。

Offset・Fillet Shape・Divisions と体積



Divisions 4 で 0.6% 以内、16 で 0.01% 以内。

Offset	Fillet Shape	Divisions	点	面	体積	丸めた箱の式	比	平らな面取りの式	秒
0.05	solid	1	24	26	0.985667	0.993729	0.99189	0.985667	0.0008
0.05	round	1	24	26	0.985667	0.993729	0.99189	0.985667	0.0002
0.05	round	2	56	54	0.991514	0.993729	0.99777	0.985667	0.0005

Offset	Fillet Shape	Divisions	点	面	体積	丸めた箱の式	比	平らな面取りの式	秒
0.05	round	4	152	150	0.993193	0.993729	0.99946	0.985667	0.0004
0.05	round	8	488	486	0.993602	0.993729	0.99987	0.985667	0.0006
0.05	round	16	1,736	1,734	0.993702	0.993729	0.99997	0.985667	0.0010
0.1	solid	1	24	26	0.945333	0.975587	0.96899	0.945333	0.0002
0.1	round	1	24	26	0.945333	0.975587	0.96899	0.945333	0.0002
0.1	round	2	56	54	0.967260	0.975587	0.99146	0.945333	0.0003
0.1	round	4	152	150	0.973708	0.975587	0.99807	0.945333	0.0004
0.1	round	8	488	486	0.975176	0.975587	0.99958	0.945333	0.0005
0.1	round	16	1,736	1,734	0.975521	0.975587	0.99993	0.945333	0.0008
0.2	solid	1	24	26	0.802667	0.907705	0.88428	0.802667	0.0003
0.2	round	1	24	26	0.802667	0.907705	0.88428	0.802667	0.0002
0.2	round	2	56	54	0.878670	0.907705	0.96801	0.802667	0.0003
0.2	round	4	152	150	0.902333	0.907705	0.99408	0.802667	0.0004
0.2	round	8	488	486	0.906848	0.907705	0.99906	0.802667	0.0005
0.2	round	16	1,736	1,734	0.907782	0.907705	1.00008	0.802667	0.0009

わかったこと

- Offset は丸みの半径。Round の体積は、その半径で丸めた箱の式に近づく。
- Divisions 1 の Round は、ただの面取り（Solid）。Divisions 4 で、丸めた箱の式との差が 0.6% 以内になった。
- 面取りした箱の体積は $1 - 6d^2 + (16/3)d^3$ 。d = 0.1 で 0.945333。

次にやること

- Offset 0.2 で式を超えた理由（角のパッチの形）
- Profile Ramp で丸み以外の形にしたときの体積

polyfill

の四角形の塞ぎ方は、辺が奇数の穴を塞がない —
三角形は 1つの穴に $n-2$ 枚、Grid は少しふくらむ

2026-09-24

何を作ったか

蓋のない tube（半径1・高さ1・Columns = n ）の上下の穴を polyfill で塞ぎ、Fill Mode
ごとに足された面を数えた。穴は正 n 角形なので、塞いだ面積は2つで $n \cdot \sin(2\pi/n)$ （ $n = 12$ で
6）のはず。

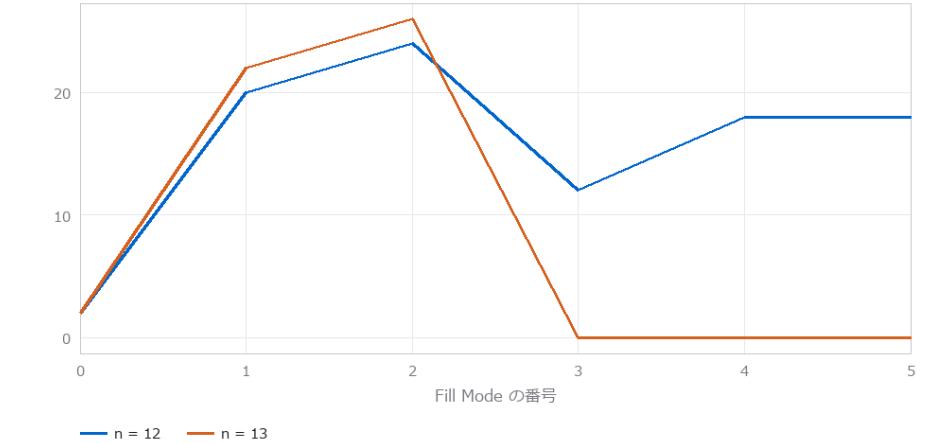
** $n = 12$ （偶数）では、6通りとも穴が塞がった。*1つの穴あたり Single Polygon は 1 枚、Triangles
は $n-2$ 枚（計 20）、Triangle Fan は中心に点を1つ足して n 枚、Quadrilateral Fan は $n/2$
枚、Quadrilaterals は 9 枚（点を 4 個足す）。面積は Grid 以外の5通りで
6.000000、体積は正12角柱の 3.000000 とぴったり合った。

** $n = 13$ （奇数）では、四角形の3通りが何も足さなかった。*Single・Triangles・Triangle Fan
は塞がる。ノードには「Some patches were not created because they contain an odd number of
edges」という警告が出る（エラーではないので、ネットワークを流れる形は穴があいたまま進む）。

**Quadrilateral Grid は、蓋が外へふくらむ。*既定（Smooth 入り・強さ 50）で上の蓋の点が $y =$
0.50496（縁は 0.5）まで出て、面積 6.000172、体積 3.014671（+0.49%）。Smooth
を切るとさらにふくらみ、 $y = 0.560072$ ・面積 6.033112 になった。

Fill Mode ごとの塞がり方

polyfill: 上下2つの穴を塞いだ面の数（0 Single / 1 Tris / 2 Tri Fan / 3 Quad Fan
足された面の数



$n = 13$ では Quad Fan・Quads・Grid が 0 枚。

n	Fill Mode	足した面	足した点	面の形	蓋の面積	式	体積	正 n 角柱の式	上下の広がり	警告	秒
12	Single Polygon	2	0	12角形 2	6.000000	6.000000	3.000000	3.000000	1	—	0.0002
12	Triangles	20	0	3角形 20	6.000000	6.000000	3.000000	3.000000	1	—	0.0001

n	Fill Mode	足した面	足した点	面の形	蓋の面積	式	体積	正 n 角柱の式	上下の広がり	警告	秒
12	Triangle Fan	24	2	3角形 24	6.000000	6.000000	3.000000	3.000000	1	—	0.0001
12	Quadrilateral Fan	12	2	4角形 12	6.000000	6.000000	3.000000	3.000000	1	—	0.0001
12	Quadrilaterals	18	8	4角形 18	6.000000	6.000000	3.000000	3.000000	1	—	0.0003
12	Quadrilateral Grid	18	8	4角形 18	6.000172	6.000000	3.014671	3.000000	1.00992	—	0.0002
13	Single Polygon	2	0	13角形 2	6.041401	6.041401	3.020701	3.020701	1	—	0.0001
13	Triangles	22	0	3角形 22	6.041401	6.041401	3.020701	3.020701	1	—	0.0001
13	Triangle Fan	26	2	3角形 26	6.041401	6.041401	3.020701	3.020701	1	—	0.0001
13	Quadrilateral Fan	0	0	—	0.000000	6.041401	2.013800	3.020701	0	あり	0.0004
13	Quadrilaterals	0	0	—	0.000000	6.041401	2.013800	3.020701	0	あり	0.0001
13	Quadrilateral Grid	0	0	—	0.000000	6.041401	2.013800	3.020701	0	あり	0.0001

Quadrilateral Grid の Smooth (n = 12)

Smooth	強さ	蓋の面積	上の蓋の y (最小～最大)
入	50	6.000172	0.5～0.50496
切	50	6.033112	0.5～0.560072

わかったこと

- ・ 奇数の穴に四角形の塞ぎ方を使うと、穴のまま進む。警告は出るが、止まらない。
- ・ 平らに塞ぎたいなら Single Polygon ・ Triangles ・ Triangle Fan ・ Quadrilateral Fan ・ Quadrilaterals。Grid だけ中がふくらむ。
- ・ Triangles は 1つの穴に n-2 枚。divide (実験125) の三角形分割と同じ枚数。

次にやること

- ・ Grid のふくらみを抑えるつまみ (Tangent Strength など)
- ・ 平らでない穴を塞いだときの形

scatter の density 属性は点を $k/(1+k)$ に分ける — 数を決めないと Density Scale × 面積のあたりで揺れる

2026-09-24

何を作ったか

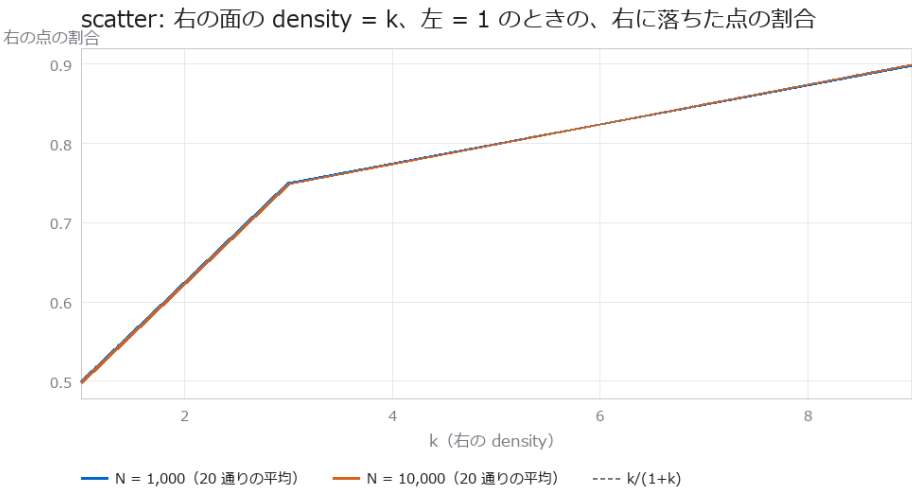
2×1 の板を左右2枚の面に分け、右の面の density を k 、左を 1 にして、Force Total Count = N で撒いた。Global Seed を 20 通り変え、右に落ちた点の割合を数えた（Relax は切った）。

**割合は $k/(1+k)$ に合った。 **6通りで、20 通りの平均と式の差は最大 0.0024。 density は「面積あたりの濃さ」として、面ごとに点を配っている。

**ばらつきは、二項分布の幅と同じか少し小さい。 **点の数の標準偏差 ÷ $\sqrt{(N \cdot p \cdot (1-p))}$ は 0.68~1.01。完全にでたらめに配るより、少しだけ揃う傾向がある（ばらつきの推定は 20 通りなので、±16% ほどの幅がある）。

**Force Total Count を切ると、数はおおよそ Density Scale × 面積。 **ぴったりではない: Density Scale 10 で 14 (20) 、Density Scale 100 で 193 (200) 、Density Scale 1000 で 1,976 (2000) 。決まった数が要るなら Force Total Count を入れる。

density の比と、右に落ちた割合（Force Total Count 入り）



点線 $k/(1+k)$ に、 $N = 1,000$ も $10,000$ も乗る。

k	N	$k/(1+k)$	平均	最小～最大	点の数の標準偏差	二項分布の幅	比	1回の秒
1	1,000	0.5000	0.4991	0.4770～0.5230	15.2	15.8	0.96	0.0038
1	10,000	0.5000	0.4985	0.4913～0.5101	44.8	50.0	0.90	0.0003
3	1,000	0.7500	0.7500	0.7230～0.7760	13.8	13.7	1.01	0.0009
3	10,000	0.7500	0.7493	0.7431～0.7561	39.7	43.3	0.92	0.0003

k	N	$k/(1+k)$	平均	最小～最大	点の数の標準偏差	二項分布の幅	比	1回の秒
9	1,000	0.9000	0.8976	0.8770～0.9140	8.7	9.5	0.91	0.0008
9	10,000	0.9000	0.8990	0.8956～0.9043	20.5	30.0	0.68	0.0003

Force Total Count を切ったとき（面積 2、Seed 0）

Density Scale	点の数	Density Scale × 面積
10	14	20
100	193	200
1000	1,976	2000

わかったこと

- density は比で効く。右を 3 倍にすれば、右に 3/4。
- 点の数を決めたいなら Force Total Count。切ると Density Scale × 面積のあたりで毎回揺れる。
- Seed を変えたときの揺れは $\sqrt{(N \cdot p \cdot (1-p))}$ くらい。N = 10,000 ・ 半々なら ±50 点ほど。

次にやること

- Relax を入れたときに割合が変わるか
- density を点の属性（頂点ごと）で与えたときの分け方

polywire の管は Wire Radius に内接する正 n 角柱で両端に蓋 — 折れ目は Prevent Joint Buckling で $r/\cos(\theta/2)$ に広がる

2026-09-24

何を作ったか

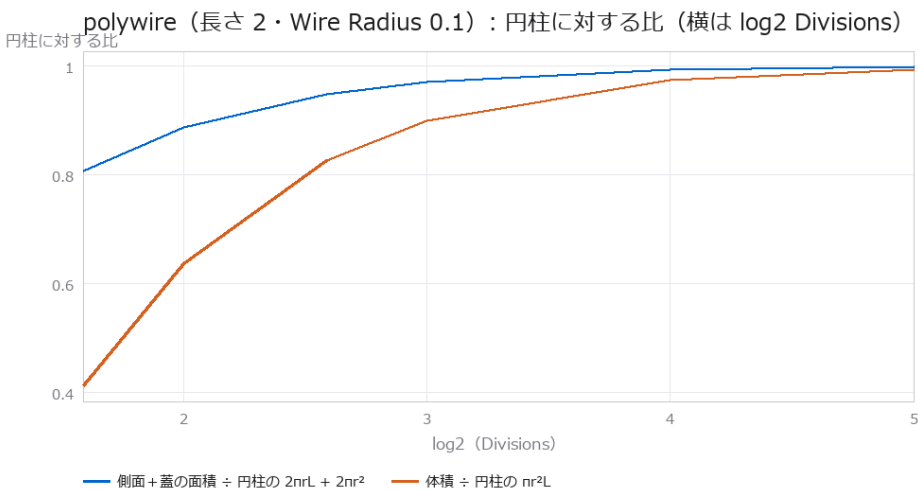
長さ 2 のまっすぐな線と、 90° に折れた線に polywire (Wire Radius $r = 0.1$) をかけ、点の位置・面積・体積を測った。

**まっすぐな線では、管は半径 r の円に内接する正 n 角柱。 **点はどれも軸からちょうど 0.1。面の数は Divisions + 2 で、両端に n 角形の蓋が付く (閉じた形)。面積は 側面 $L \cdot 2n \cdot r \cdot \sin(n/n) +$ 蓋 $2 \cdot (n/2) r^2 \cdot \sin(2n/n)$ 、体積は $L \cdot (n/2) r^2 \cdot \sin(2n/n)$ と、6通りとも合った (差は最大 $2e-16$)。

**折れ目では、Prevent Joint Buckling (既定で入) が輪を曲がりの面の中だけ広げる。 ** 90° の折れ目の輪は、曲がりの面の中で $0.141421 (= r/\cos 45^\circ = 0.141421)$ 、面に垂直な向きは 0.1 のまま。輪は2本の線の間の、ちょうど半分の角度の面に乗っている。管の太さが折れ目でも変わらないようにする仕組み。

**切ると、折れ目の輪は半径 r の円のまま (0.1)。 **曲がりの内側で管が細くくびれる。面積は $1.307150 \rightarrow 1.220392$ に減った。

まっすぐな線 (長さ 2)



内接多角形なので、どちらも 1 より下から近づく。

Divisions	点	面	軸からの距離	面積	側面+蓋の式	体積	式	秒
3	6	5	0.1~0.1	1.065211	1.065211	0.025981	0.025981	0.0004
4	8	6	0.1~0.1	1.171371	1.171371	0.040000	0.040000	0.0002
6	12	8	0.1~0.1	1.251962	1.251962	0.051962	0.051962	0.0002
8	16	10	0.1~0.1	1.281156	1.281156	0.056569	0.056569	0.0002
16	32	18	0.1~0.1	1.309807	1.309807	0.061229	0.061229	0.0002
32	64	34	0.1~0.1	1.317048	1.317048	0.062429	0.062429	0.0002

90° の折れ目の輪 (Divisions 16)

Prevent Joint Buckling	輪の点	折れ目からの距離 (最小～最大)	曲がりの面の中の広がり	$r/\cos 45^\circ$	二等分面からのずれ	全体の面積
入	16	0.1～0.141421	0.141421	0.141421	0	1.307150
切	16	0.1～0.1	0.1	0.141421	0	1.220392

わかったこと

- ・ polywire の管は内接多角形。Divisions 4 だと断面積は円の 64%。
- ・ 両端には蓋が付く。体積をそのまま measure で測れる。
- ・ 折れ目で細くなるなら、Prevent Joint Buckling が切れていないか。入れると $r/\cos(\theta/2)$ に広がって太さを保つ。

次にやること

- ・ 急な折れ目 (150° など) で Max Joint Scale が効くところ
- ・ Segments を増やしたときの点の数

uvlayout は長方形 12 枚で升の 48~84% を埋める — Padding は Search Resolution が粗いほど大きく効く

2026-09-24

何を作ったか

大きさの違う長方形 12 枚（辺 0.4~2.0）に形そのままの UV を付けて uvlayout にかけて、並べたあとの UV の面積の合計（0~1 の升が埋まった割合）を測った。Island Padding・Island Rotation Step・Search Resolution を変えた。

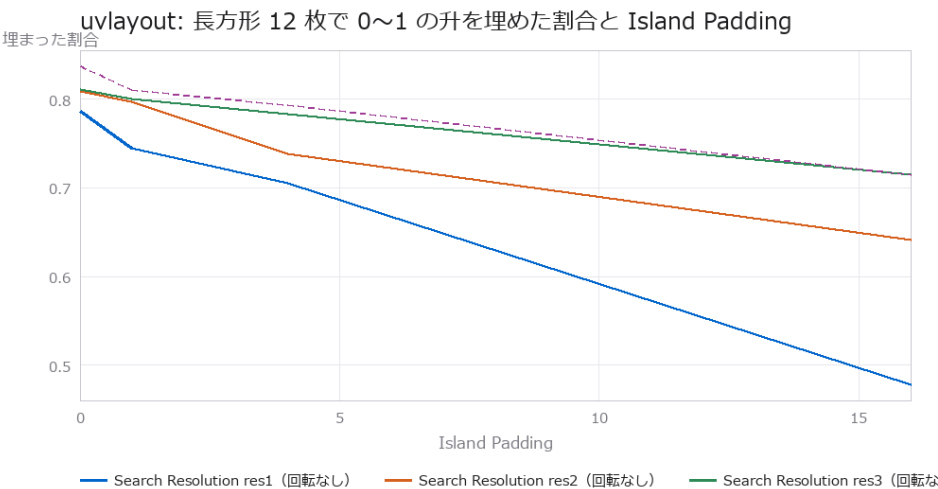
**島どうしの大きさの比は保たれた。 **全24通りで、元の面積に対する UV の面積の比は、島の間で最大 1.000001 倍しか変わらない。どの島も同じ倍率で縮めて並べている。

**埋まった割合は 48~84%。 **いちばん良かったのは 90° 回転あり・Padding 0・res3 の 83.7%。回転なしでも同じ条件で 81.1%。長方形でも、回していい向きがあると少し詰まる。

**Padding は Search Resolution が粗いほど大きく効く。 **Padding 16 で、res1 は 47.8%、res3 は 71.5%。Padding が「探す升目の何個分」で決まっているように見えるが、確かめていない。

**速い。 **どれも 0.016 秒以下（12 枚）。細かい res3 でも数ミリ秒。

Rotation Step・Padding・Search Resolution と埋まった割合



res1（青）は Padding を上げると急に落ちる。

Rotation Step	Padding	Search Resolution	埋まった割合	島の倍率の最大比	U の範囲	V の範囲	秒
none	0	res1	78.6%	1.000000	0~0.9699	0~0.9902	0.003
none	0	res2	80.9%	1.000000	0~0.9829	0~0.9998	0.003
none	0	res3	81.1%	1.000000	0~0.9851	0~1	0.006
none	1	res1	74.5%	1.000000	0~0.999	0~0.9842	0.002
none	1	res2	79.7%	1.000000	0~0.9792	0~0.9988	0.003
none	1	res3	80.0%	1.000000	0~0.9797	0~0.9972	0.005

Rotation Step	Padding	Search Resolution	埋まった割合	島の倍率の最大比	U の範囲	V の範囲	秒
none	4	res1	70.5%	1.000001	0~0.9991	0~0.9999	0.002
none	4	res2	73.9%	1.000000	0~1	0~0.9934	0.003
none	4	res3	78.3%	1.000000	0~0.9743	0~0.9968	0.005
none	16	res1	47.8%	1.000001	0~1	0~0.9712	0.002
none	16	res2	64.1%	1.000001	0~0.9676	0~0.9953	0.003
none	16	res3	71.5%	1.000000	0~0.9999	0~0.9999	0.005
PI2	0	res1	82.8%	1.000001	0~0.9954	0~0.9921	0.002
PI2	0	res2	82.8%	1.000001	0~0.9954	0~0.9862	0.005
PI2	0	res3	83.7%	1.000000	0~0.9996	0~0.9908	0.013
PI2	1	res1	78.6%	1.000000	0~0.9573	0~0.9999	0.002
PI2	1	res2	79.7%	1.000000	0~0.9792	0~0.9988	0.005
PI2	1	res3	81.1%	1.000000	0~0.9871	0~0.9791	0.013
PI2	4	res1	70.5%	1.000001	0~0.9991	0~0.9999	0.002
PI2	4	res2	74.5%	1.000000	0~0.9828	0~0.996	0.005
PI2	4	res3	79.4%	1.000001	0~0.9611	0~0.9986	0.016
PI2	16	res1	54.2%	1.000000	0~0.9883	0~0.9921	0.002
PI2	16	res2	64.1%	1.000000	0~0.9969	0~0.8956	0.004
PI2	16	res3	71.5%	1.000000	0~0.9999	0~0.9999	0.012

わかったこと

- ・島の大きさの比は変えない。1 枚だけ大きくしたいなら、先に UV を拡大しておく（島ごとの倍率の属性もある）。
- ・Padding を大きくするなら Search Resolution も上げる。粗いままだと升の半分しか使わない。
- ・長方形の島でも 90° 回転を許すと数 % 詰まる。

次にやること

- ・Island Padding と、書き出す画像のピクセルの関係
- ・形の複雑な島（L 字など）での埋まり方

attribtransfer は Distance Threshold までそのまま運び、Blend Width の外側で $(1 - t^2)^2$ に落とす

2026-09-24

何を作ったか

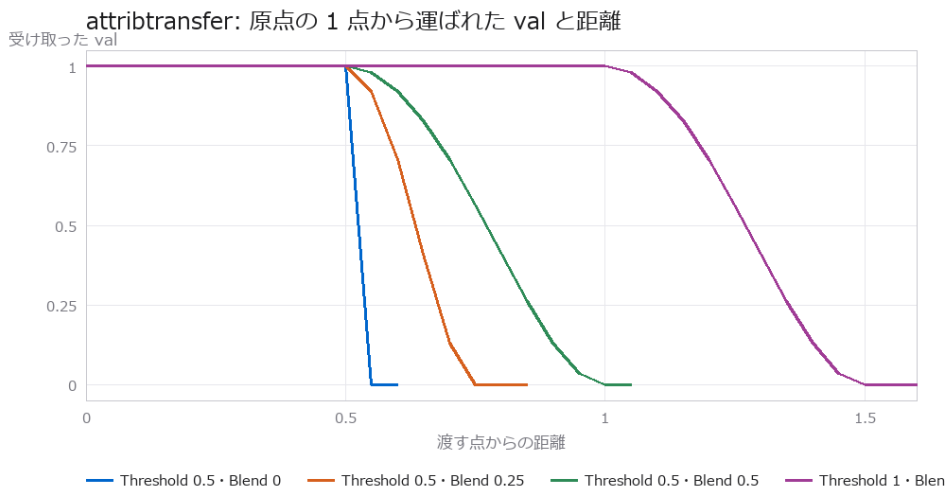
x = 0~2 に 0.01 刻みで並べた 201 点へ、原点の 1 点から val = 1 を attribtransfer で運んだ。Distance Threshold (d) と Blend Width (b) を変え、距離ごとに受け取った値を読んだ。

**d までは 1 のまま、d から d + b で 0 まで下がる。 **弱まる区間は Threshold の内側ではなく外側に付く (d = 0.5・b = 0.25 で、0.5 まで 1、0.74 まで何か届き、0.75 から 0)。Blend Width = 0 なら d でぶつりと切れる。

**下がり方は直線ではなく $(1 - t^2)^2$ (t = (距離 - d)/b)。 **記録した点すべてで、この式との差は最大 0.0000。始まりはゆっくり、真ん中で急に、終わりはまたゆっくり 0 へ着く。t = 0.5 で 0.5625。

**時間はほとんどかからない。 **1回目だけ 0.09 秒 (最初の組み立て)、あとは 0.0001 秒。

Threshold・Blend と、値の届き方



どの線も Threshold までは 1。Blend の分だけ外へなだらかに伸びる。

Distance Threshold	Blend Width	1 のままの所まで	届く所まで	中間の点	直線との差 (最大)	秒
0.5	0	0.5	0.5	0	—	0.0868
0.5	0.25	0.5	0.74	24	0.1293	0.0001
0.5	0.5	0.5	0.99	49	0.1294	0.0001
1	0	1	1	0	—	0.0001
1	0.25	1	1.24	24	0.1293	0.0001
1	0.5	1	1.49	49	0.1294	0.0001

わかったこと

- Blend Width は Threshold の外に足される。届く距離は d + b。

- ・弱まり方は $(1 - t^2)^2$ 。直線より、端がなめらか。
- ・ここまでの結果は、渡す点が 1 つのとき。Max Sample Count を上げて複数の点を混ぜるときは別の話。

次にやること

- ・Max Sample Count を上げたときの混ざり方 (Kernel Function)
- ・面 (primitive) から運ぶときの距離の測り方

subdivide の crease は「重み w の回数までは尖ったまま」 — $w \geq \text{Iterations}$ なら箱はそのまま、体積 1

2026-09-24

何を作ったか

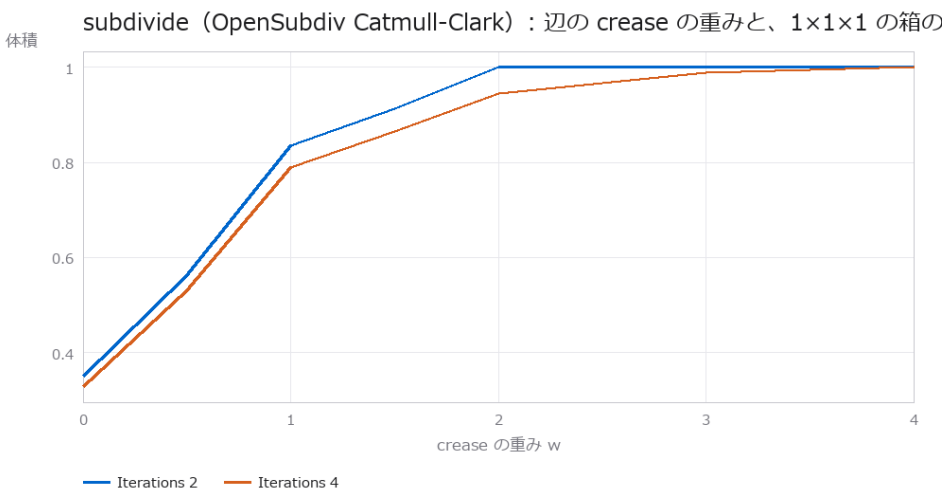
1×1×1 の箱の辺すべてに crease SOP (Operation = Set) で重み w を付け、subdivide (Algorithm = OpenSubdiv Catmull-Clark) で Iterations 2 回と 4 回の細分をかけた。体積と、角の点が元の角 (0.5, 0.5, 0.5) からどれだけ離れたかを測った。

**重みが細分の回数以上なら、形は元の箱のまま。 **体積 1.000000、角のずれ 0 になったのは、 $w \geq \text{Iterations}$ のときだけ (16通りすべてこの通り)。 $w = 2$ は Iterations 2 で箱のまま、4 では 0.945526 に丸まる。重みは「何回目の細分まで尖らせるか」の回数として効いている。

**重み 0 はただの Catmull-Clark。 **Iterations 4 で体積 0.328926、角は 0.433 内側へ下がる。

**重みの小数も効く。 ** $w = 0.5 \cdot 1.5$ は、その前後の整数の間の丸さになった (Iterations 4 で $0.529 \cdot 0.865$)。 $w \geq 1$ になると、面の真ん中は元の面の上 (外へ 0.5) まで届く。

crease の重み・Iterations と形



Iterations 2 (青) は $w = 2$ で体積 1 に届き、4 (橙) は $w = 4$ で届く。

重み w	Iterations	点	面	体積	角のずれ	いちばん外の座標	秒
0	2	98	96	0.350192	0.424994	0.439236	0.0008
0	4	1,538	1,536	0.328926	0.432790	0.420959	0.0012
0.5	2	98	96	0.561682	0.308722	0.469618	0.0003
0.5	4	1,538	1,536	0.529197	0.324314	0.46048	0.0009
1	2	98	96	0.834635	0.192450	0.5	0.0003
1	4	1,538	1,536	0.789774	0.215838	0.5	0.0008
1.5	2	98	96	0.913194	0.096225	0.5	0.0003

重み w	Iterations	点	面	体積	角のずれ	いちばん外の座標	秒
1.5	4	1,538	1,536	0.864630	0.161043	0.5	0.0013
2	2	98	96	1.000000	0.000000	0.5	0.0003
2	4	1,538	1,536	0.945526	0.106248	0.5	0.0008
3	2	98	96	1.000000	0.000000	0.5	0.0003
3	4	1,538	1,536	0.988627	0.048113	0.5	0.0008
4	2	98	96	1.000000	0.000000	0.5	0.0003
4	4	1,538	1,536	1.000000	0.000000	0.5	0.0009
10	2	98	96	1.000000	0.000000	0.5	0.0003
10	4	1,538	1,536	1.000000	0.000000	0.5	0.0010

わかったこと

- ・crease の重みは「細分の回数」。Iterations を増やしたら、同じ硬さに見せるには重みも増やす。
- ・完全に尖らせたいなら、重みを Iterations 以上に。10 のような大きな値なら回数を気にしなくてよい。
- ・重み 0 の箱は、体積が $1/3$ ほどまで縮む。形を保ちたい所には crease か、辺を足す。

次にやること

- ・角の点（頂点）の crease の効き方
- ・Houdini Catmull-Clark (algorithm = houdini) との違い

vdbfromparticles の粒は pscale を半径にした球 — 2つ重ねると「2つの球の和」の体積に、升目の2乗で近づく

2026-09-24

何を作ったか

pscale = 0.5 の2点を間隔 s に置き、vdbfromparticles（距離の VDB）→ convertvdb（Polygons）で面にして体積を測った。比べる式は、半径 r の球2つを間隔 s で重ねた体積 $2 \cdot (4/3)\pi r^3 - \pi(4r + s)(2r - s)^2/12$ 。

**粒の半径は pscale そのまま。 **間隔 0（1つの球）で、囲む箱の幅は 1 に近づいた（Voxel 0.0125 で 0.9997）。間隔 1.2（離れた2つ）では横幅がちょうど 2.2。

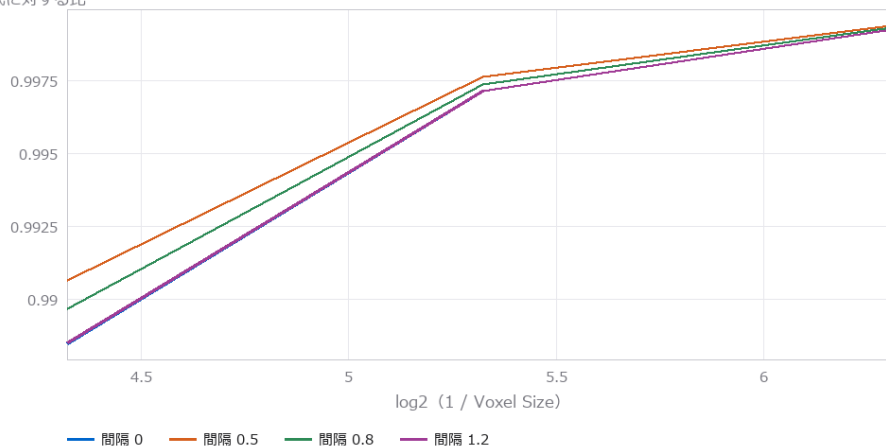
**重なった2つの粒の体積は、2つの球の和の式に合う。 **Voxel 0.0125 で、4通りとも式の 99.93~99.94%。重なった部分でふくらんだり、なめらかに埋まったりはしない（尖った和のまま）。

**いつも少し小さく、Voxel を半分にすると不足が約 1/4 になる。 **不足の縮み方は 間隔 0 で 4.01・3.99 倍、間隔 0.5 で 3.96・3.93 倍、間隔 0.8 で 3.95・4.03 倍、間隔 1.2 で 4.01・3.99 倍。Voxel 0.05 で約 1%、0.025 で 0.3%、0.0125 で 0.07%。

面の数は Voxel を半分にするとほぼ4倍（間隔 0 で 1,830 → 30,078）。時間はどれも 0.01 秒前後（1回目だけ 0.09 秒）。

間隔・Voxel Size と体積

vdbfromparticles（半径 0.5 の粒 2 つ）：体積 ÷ 2つの球の和の式（横は $\log_2(1/\text{Voxel Size})$ 式に対する比



4本の線がほぼ重なる。重なり方によらず、升目の細かさだけで決まる。

間隔 s	Voxel Size	体積	2つの球の和	比	横幅	式の横幅	縦	面	秒
0	0.05	0.517568	0.523599	0.98848	0.995	1	0.995	1,830	0.087
0	0.025	0.522098	0.523599	0.99713	0.9987	1	0.9987	7,470	0.004
0	0.0125	0.523224	0.523599	0.99928	0.9997	1	0.9997	30,078	0.009

間隔 s	Voxel Size	体積	2つの球の和	比	横幅	式の横幅	縦	面	秒
0.5	0.05	0.875323	0.883573	0.99066	1.495	1.5	0.995	2,710	0.014
0.5	0.025	0.881490	0.883573	0.99764	1.4987	1.5	0.9987	11,102	0.004
0.5	0.0125	0.883046	0.883573	0.99940	1.4997	1.5	0.9997	44,798	0.010
0.8	0.05	1.007350	1.017876	0.98966	1.795	1.8	0.995	3,326	0.014
0.8	0.025	1.015214	1.017876	0.99738	1.7987	1.8	0.9987	13,566	0.004
0.8	0.0125	1.017214	1.017876	0.99935	1.7997	1.8	0.9997	54,510	0.009
1.2	0.05	1.035145	1.047198	0.98849	2.2	2.2	0.995	3,700	0.013
1.2	0.025	1.044196	1.047198	0.99713	2.2	2.2	0.9987	14,980	0.004
1.2	0.0125	1.046448	1.047198	0.99928	2.2	2.2	0.9997	60,196	0.010

わかったこと

- ・粒の大きさは pscale（半径）で決まる。直径ではない。
- ・重なりは尖った和。くっついた所をなめらかにしたいなら、あとで vdbsmoothsdf などかける（確かめていない）。
- ・Voxel Size を半分にすると、体積の不足は約 1/4、面は約 4 倍。

次にやること

- ・Minimum Radius in Voxels より小さい粒の扱い
- ・たくさんの粒（液体）での体積の合計

smooth の縮み方は $r = 1/(1 + \text{Strength} \cdot L^q / C(2q, q)) - 36$ 通りで6桁一致。Filter Quality を上げるほど形を保つ

2026-09-24

何を作ったか

半径 1 の circle (Polygon、n 角形) に smooth (smooth::2.0、Method = Uniform、Constrained Boundary なし) をかけ、残った半径 r を測った。点の数 n ・ Strength S ・ Filter Quality q を変えた (36 通り)。

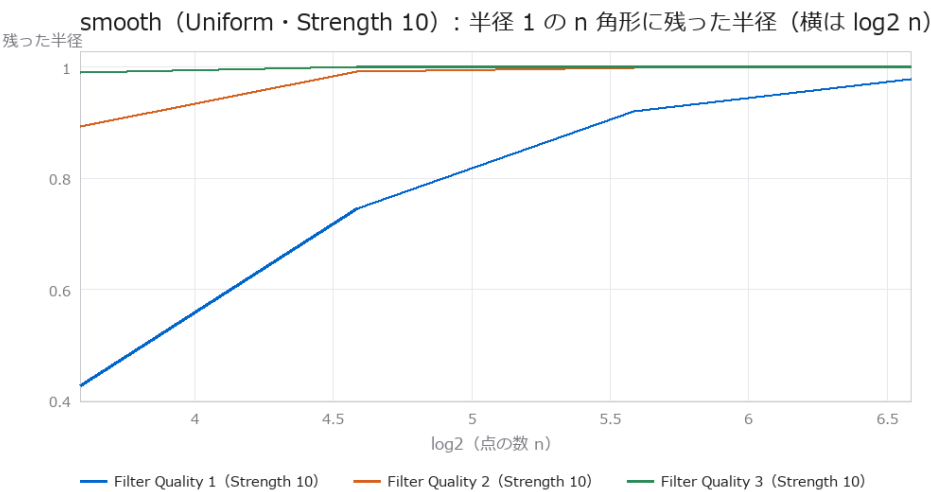
****Filter Quality 1 では $r = 1/(1 + S \cdot (1 - \cos \theta))$ ($\theta = 2\pi/n$) 。******n = 12 ・ S = 10 で 0.4273968 (式 0.4273968) 。**となりの平均へ近づける計算を、1 回で「行き先から逆算して」解く形 (陰的な1ステップ) になっている。S をいくら大きくしても、半径は 0 にはならない。

****Filter Quality q を上げると、 $r = 1/(1 + S \cdot L^q / C(2q, q))$ 。******L = 2(1 - cos θ) (となりとの差の強さ) 、C(2q, q) は二項係数 (q = 1 ・ 2 ・ 3 で 2 ・ 6 ・ 20) 。**36 通りすべてで、この式との差は最大 3e-06。ここで使った円では L は 1 より小さいので、q を上げるほど縮みにくくなる。細かいでこぼこ (L が大きい) は消し、なだらかな形 (L が小さい) は残すフィルタになっている。

****点の数が多いほど縮まない。******同じ Strength 10 ・ Quality 1 で、n = 12 は 0.427、n = 96 は 0.979。**細かい網ほど、となりとの差が小さいため。

****どの点も同じだけ縮む** (半径のばらつきは最大 1.3e-6) 。**形は円のまま小さくなる。

点の数 ・ Strength ・ Filter Quality と、残った半径



Quality 3 (緑) は n = 24 でもほぼ縮まない。

n	Strength	Filter Quality	残った半径	式	$-\ln r \div (1 - \cos \theta)$	点ごとの差	秒
12	1	1	0.8818540	0.8818540	0.93845	2.0e-07	0.0004

n	Strength	Filter Quality	残った半径	式	$-\ln r \div (1 - \cos \theta)$	点ごとの差	秒
12	1	2	0.9881754	0.9881754	0.08879	2.0e-07	0.0002
12	1	3	0.9990390	0.9990390	0.00718	2.0e-07	0.0002
12	10	1	0.4273968	0.4273968	6.3448	6.0e-07	0.0002
12	10	2	0.8931273	0.8931272	0.84364	8.0e-07	0.0001
12	10	3	0.9904727	0.9904727	0.07145	1.3e-06	0.0001
12	100	1	0.0694567	0.0694567	19.9072	2.0e-07	0.0001
12	100	2	0.4552464	0.4552464	5.87363	5.0e-07	0.0001
12	100	3	0.9122511	0.9122511	0.6855	4.0e-07	0.0001
24	1	1	0.9670486	0.9670486	0.98334	1.0e-07	0.0001
24	1	2	0.9992266	0.9992266	0.02271	2.0e-07	0.0001
24	1	3	0.9999842	0.9999842	0.00046	2.0e-07	0.0001
24	10	1	0.7458558	0.7458558	8.60543	1.3e-06	0.0001
24	10	2	0.9923191	0.9923191	0.22629	5.0e-07	0.0001
24	10	3	0.9998420	0.9998418	0.00464	6.0e-07	0.0001
24	100	1	0.2268902	0.2268902	43.5312	4.0e-07	0.0001
24	100	2	0.9281576	0.9281576	2.18799	9.0e-07	0.0001
24	100	3	0.9984200	0.9984200	0.0464	9.0e-07	0.0001
48	1	1	0.9915174	0.9915174	0.99575	1.0e-07	0.0001
48	1	2	0.9999512	0.9999512	0.0057	1.0e-07	0.0002
48	1	3	1.0000000	0.9999997	0	1.0e-07	0.0001
48	10	1	0.9211908	0.9211908	9.59517	3.0e-07	0.0004
48	10	2	0.9995124	0.9995123	0.05701	5.0e-07	0.0002
48	10	3	1.0000000	0.9999975	0	1.0e-07	0.0001
48	100	1	0.5389343	0.5389343	72.2562	3.0e-07	0.0001
48	100	2	0.9951443	0.9951443	0.56896	6.0e-07	0.0002
48	100	3	0.9999762	0.9999750	0.00278	9.0e-07	0.0002
96	1	1	0.9978635	0.9978635	0.99893	1.0e-07	0.0001
96	1	2	1.0000000	0.9999969	0	1.0e-07	0.0001
96	1	3	1.0000000	1.0000000	0	1.0e-07	0.0002
96	10	1	0.9790380	0.9790380	9.89445	2.0e-07	0.0002
96	10	2	0.9999707	0.9999694	0.01366	6.0e-07	0.0002
96	10	3	1.0000000	1.0000000	0	1.0e-07	0.0003
96	100	1	0.8236502	0.8236502	90.613	3.0e-07	0.0001
96	100	2	0.9996945	0.9996945	0.14269	4.0e-07	0.0002
96	100	3	1.0000000	0.9999996	0	1.0e-07	0.0002

わかったこと

- ・ smooth は Strength をいくら上げても、ゼロまでは縮まない。 $1/(1 + S \cdots)$ の形。
- ・ 形を保ったまま細かいでこぼこだけ消したいなら Filter Quality を上げる。3 なら 96 角形の円はまったく縮まない。
- ・ 粗い網ほど縮む。同じ Strength でも、点が少ない所は形が大きく変わる。

次にやること

- ・ Scale Dominant ・ Curvature Dominant の縮み方
- ・ 開いた線（端が固定されない）での縮み方

smooth の3つの Method

は、点が等間隔の円では同じ縮み方

開いた線の端は Constrained Boundary

で止まり、波の高さは $1/(1 + S \cdot L/2)$ だけ残る

2026-09-24

何を作ったか

実験185 の続き。半径 1 の 24 角形に、Method を Uniform・Scale Dominant・Curvature Dominant と変えて smooth (Strength 10) をかけた。また、 $x = 0 \sim 1$ に 41 点で $y = 0.1 \cdot \sin(4\pi x)$ の波の線を作り、端の扱い (Constrained Boundary) を変えて smooth をかけた。

円では、3つの Method の結果が7桁まで同じだった (Quality ごとに同じ値。Quality 1 で 0.7458558、2 で 0.9923191)。点が等間隔で、どの点のまわりも同じ形なので、重みの付け方の違いが出ない。違いが出るのは、辺の長さや曲がり方がそろっていない網のとき (確かめていない)。

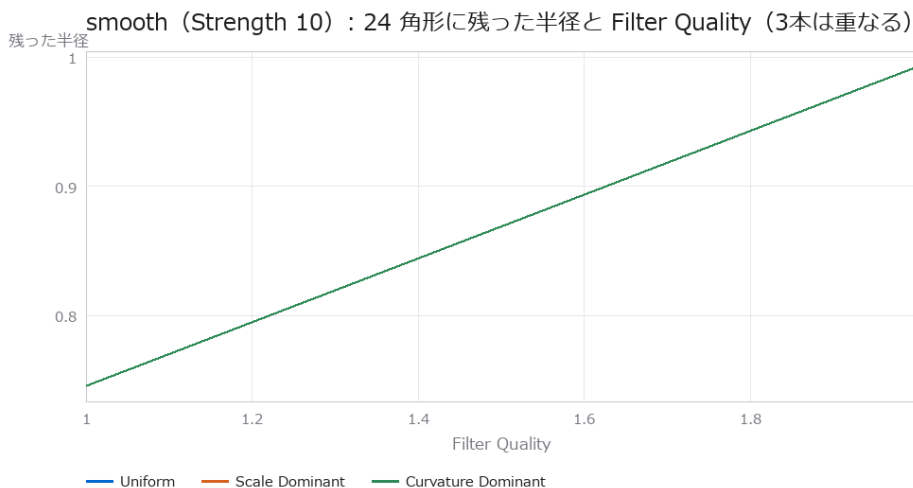
**開いた線は、Constrained Boundary = None

だと端の点も動き、線が縮む。**両端が内側へ寄り、長さ 1 の線が x 方向に 0.8909 になった。既定の Unshared Edges なら、端は (0, 0) と (1, 0) のまま。

端を止めたとき、波の高さは 67.14% 残った波1つが 20 点なので $\theta = 2\pi/20$ 、 $L = 2(1 - \cos \theta) = 0.0979$ 。実験185 の式 $1/(1 + S \cdot L/2) = 0.6714$

とぴったり合う。円の縮み方の式は、波にもそのまま使える。端も動くと、波の高さは 72.1% だった。

24 角形と Method



3つの線は完全に重なる。

Method	Filter Quality	残った半径	点ごとの差
Uniform	1	0.7458558	1.3e-06
Uniform	2	0.9923191	5.0e-07
Scale Dominant	1	0.7458558	8.0e-07
Scale Dominant	2	0.9923191	7.0e-07

Method	Filter Quality	残った半径	点ごとの差
Curvature Dominant	1	0.7458558	9.0e-07
Curvature Dominant	2	0.9923191	3.0e-07

開いた波の線 (Uniform・Strength 10・Quality 1)

Constrained Boundary	始まりの点	終わりの点	x の長さ	波の高さ	残った割合	式 $1/(1 + S \cdot L/2)$
none	(0.05455, 0.04527)	(0.94545, -0.04527)	0.89089	0.07207	72.07%	67.14%
unsharededges	(0, 0)	(1, -0)	1	0.06714	67.14%	67.14%

わかったこと

- ・ 等間隔の網なら、Method はどれでも同じ。違いが出るのは網の細かさがそろっていないとき。
- ・ 開いた線の端を動かしたくないなら、Constrained Boundary は既定の Unshared Edges のまま。None にすると線が縮む。
- ・ 波の残り方は、波1つに入る点の数で決まる。 $1/(1 + S \cdot L/2)$ 、 $L = 2(1 - \cos(2\pi/\text{点の数}))$ 。

次にやること

- ・ 点の間隔がそろっていない線での Method の違い
- ・ Filter Quality 2・3 で波の高さはどう残るか

点の間隔がそろわない円では、smooth は粗い側を大きく縮める — 3つの Method の差は小さく、どれも丸さを保たない

2026-09-24

何を作ったか

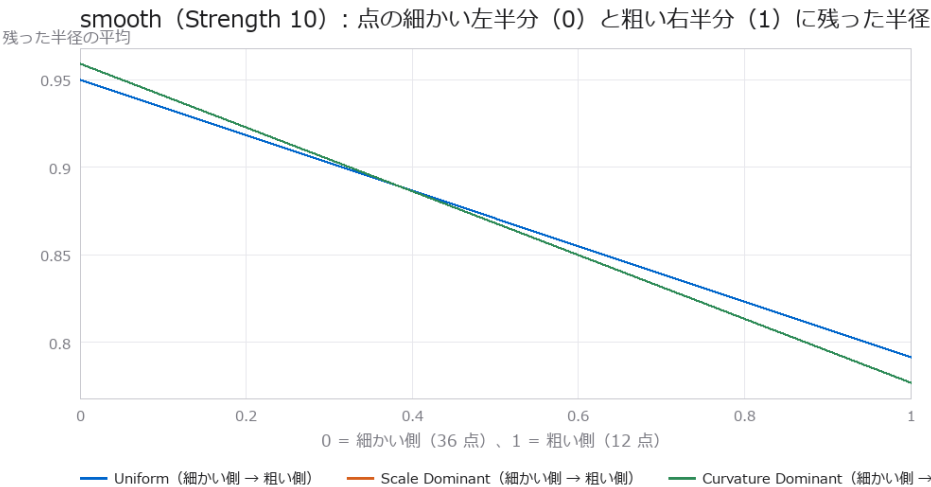
実験186 の続き。半径 1 の円の上に、左半分に 36 点・右半分に 12 点と偏らせて 48 点を置いた多角形に、Method を変えて smooth (Filter Quality 1) をかけた。

****粗い側ほど縮む。****Strength 10・Uniform で、細かい左半分の半径は 0.950、粗い右半分は 0.791。丸かった形が、半径の差 0.201 のゆがんだ形になった。点の間が広い所ほど、となりとの差 (L) が大きい (実験185・186 の式と同じ理由) 。

****Scale Dominant と Curvature Dominant は、ほぼ同じ結果**** (半径の平均 0.915098 と 0.915134) 。Uniform より点の動きが少し小さい (平均 0.0942 → 0.0864) が、左右の差はむしろ少し大きい (0.201 → 0.222) 。

****Strength 1 なら、どれも差はわずか**** (半径の差 0.029~0.029) 。

Method・Strength と形



どの Method も右 (粗い側) で大きく下がる。

Method	Strength	半径の平均	細かい側	粗い側	半径の差 (最大 - 最小)	点の動きの平均
Uniform	1	0.98912	0.99580	0.96903	0.02913	0.01268
Uniform	10	0.91003	0.95005	0.79137	0.20079	0.09424
Scale Dominant	1	0.98942	0.99606	0.96836	0.02917	0.01059
Scale Dominant	10	0.91510	0.95879	0.77718	0.22195	0.08636
Curvature Dominant	1	0.98942	0.99606	0.96835	0.02916	0.01059

Method	Strength	半径の平均	細かい側	粗い側	半径の差（最大 － 最小）	点の動きの平均
Curvature Dominant	10	0.91513	0.95884	0.77711	0.22207	0.08637

わかったこと

- ・ smooth の前に、点の間隔をそろえる。resample や remesh で細かさをそろえないと、粗い所だけ形が崩れる。
- ・ Method を変えても、この偏りはほとんど直らない。
- ・ 弱い Strength（1）なら、偏りはほぼ目立たない。

次にやること

- ・ resample でそろえてから smooth したときの形
- ・ 面（2次元の網）での Method の違い

polyextrude の Inset は上の面を i だけ内側へ寄せる — 体積は角錐台の式どおり、 $i \geq 0.5$ で四角錐になり、それ以上は裏返らない

2026-09-24

何を作ったか

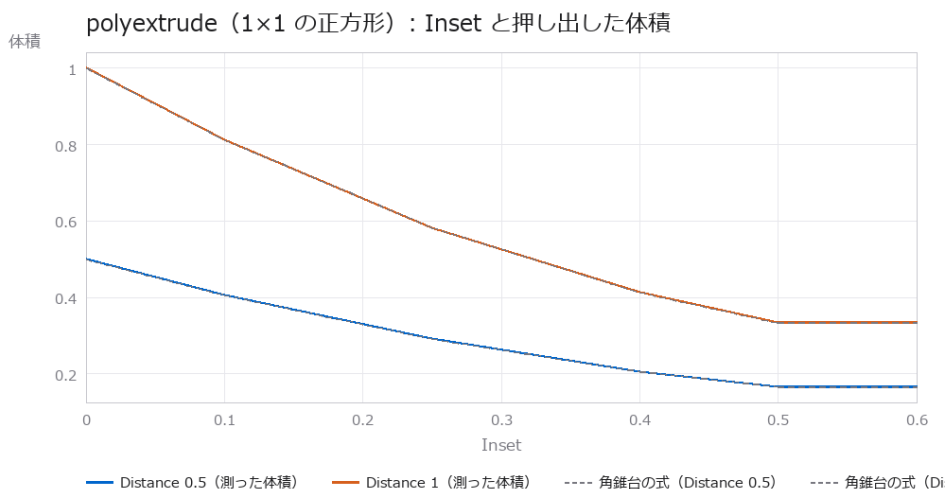
1 × 1 の正方形1枚を polyextrude (Output Back 入) で押し出し、Distance d と Inset i を変えて、上の面の大きさと体積を測った。

**上の面は、辺が $1 - 2i$ の正方形になった。 **体積は角錐台の式 $d/3 \cdot (A_1 + A_2 + \sqrt{(A_1 A_2)})$ ($A_1 = 1$ 、 $A_2 = (1 - 2i)^2$) と、 $i \leq 0.5$ の 10 通りすべてで差 $0e+00$ 以内。上の面は、Distance の高さにちょうど乗る。

** $i = 0.5$ で上の面が点になり、四角錐になる** (体積 $d/3$ 、 $d = 1$ で 0.333333) 。

** i を 0.5 より大きくしても、上の面は裏返らない。 ** $i = 0.6$ で上の面は 0.0001 の小さな正方形のまま止まり、体積は 0.333367 (四角錐とほぼ同じ)。点の数も 8 のまま (上の4点がほぼ重なる)。Inset を大きくしすぎると、見た目は尖っていても、点が重なった形になる。

Distance ・ Inset と形



測った体積 (実線) は式 (点線) に重なり、 0.5 から先は平らになる。

Distance	Inset	点	面	上の面の辺	$1 - 2i$	体積	角錐台の式	秒
0.5	0	8	6	1	1	0.500000	0.500000	0.0006
0.5	0.1	8	6	0.8	0.8	0.406667	0.406667	0.0002
0.5	0.25	8	6	0.5	0.5	0.291667	0.291667	0.0003
0.5	0.4	8	6	0.2	0.2	0.206667	0.206667	0.0001
0.5	0.5	8	6	0	0	0.166667	0.166667	0.0002
0.5	0.6	8	6	0.0001	0	0.166683	0.166667	0.0002
1	0	8	6	1	1	1.000000	1.000000	0.0001

Distance	Inset	点	面	上の面の辺	$1 - 2i$	体積	角錐台の式	秒
1	0.1	8	6	0.8	0.8	0.813333	0.813333	0.0002
1	0.25	8	6	0.5	0.5	0.583333	0.583333	0.0001
1	0.4	8	6	0.2	0.2	0.413333	0.413333	0.0001
1	0.5	8	6	0	0	0.333333	0.333333	0.0002
1	0.6	8	6	0.0001	0	0.333367	0.333333	0.0002

わかったこと

- Inset は面の縁から内側へ寄せる距離。上の面の辺は $1 - 2i$ 。
- 押し出した体積は角錐台の式で出せる。 $i = 0.25 \cdot d = 1$ で 0.583333。
- Inset を面の半分より大きくすると、上の点が重なる。尖らせたいなら、あとで fuse してまとめる。

次にやること

- 三角形・六角形の面での Inset
- Individual Elements を切ったとき（隣り合う面をまとめて押し出す）の体積

vdbreshapesdf の Dilate・Erode は、球の半径を Offset × 升の大きさだけ変える — Iterations は効かず、Open・Close は形をほぼ変えない

2026-09-24

何を作ったか

半径 1 の球を vdbfrompolygons (Voxel Size 0.02) で SDF にし、vdbreshapesdf の Operation と Offset (升目の数) を変えた。convertvdb

で面に戻して体積を測り、同じ体積の球の半径に直して比べた。

****Dilate は半径を + Offset × 0.02、Erode は − Offset × 0.02 だけ変えた。*6 通りで、ずれは最大 0.036 升 (Offset 5 の Dilate で +0.0993、Erode で -0.1006) 。SDF の値から Offset を引く (足す) だけの、面を等しく押し出す処理になっている。**

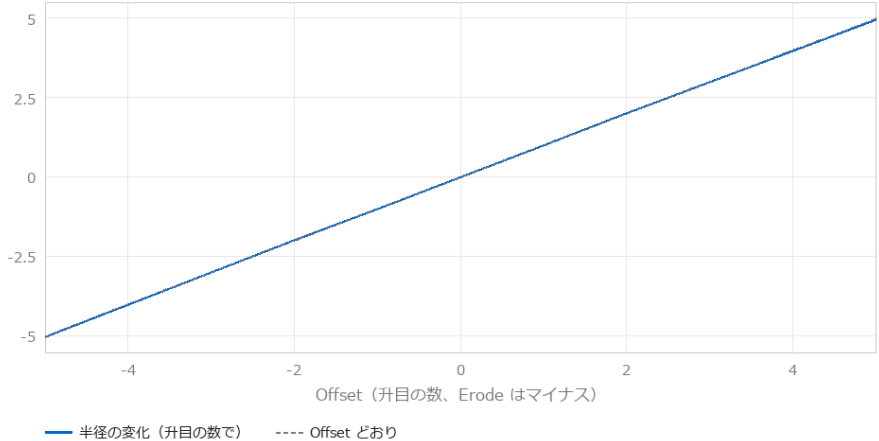
****Iterations (既定 4) を 1 にしても、結果は同じだった** (すべて一致) 。Dilate・Erode には効かないつまみとみられる。**

****Open (Erode してから Dilate) と Close (その逆) は、球をほぼ元のまま残した** (半径の変化 -0.022・-0.022 升) 。丸い形には削れる角がないため。尖った角のある形で差が出るはず (確かめていない) 。**

****時間は 1 回 0.5 秒前後** (SDF にするところと面に戻すところを含む) 。**

Operation・Offset・Iterations と半径

vdbreshapesdf (半径 1 の球・Voxel 0.02) : Offset (Erode は負) と、半径の変
半径の変化 ÷ 升の大きさ



測った点は点線 (Offset どれ) にほぼ乗る。

Operation	Offset (升)	Iterations	半径	変化	Offset × 升の大きさ	変化 ÷ 升	秒
(なし)	0	0	0.999696	+0.000000	+0.0000	+0.000	0.000
dilate	1	4	1.019624	+0.019928	+0.0200	+0.996	0.533
dilate	1	1	1.019624	+0.019928	+0.0200	+0.996	0.517
dilate	2	4	1.039501	+0.039805	+0.0400	+1.990	0.567
dilate	2	1	1.039501	+0.039805	+0.0400	+1.990	0.541

Operation	Offset (升)	Iterations	半径	変化	Offset × 升の大きさ	変化 ÷ 升	秒
dilate	5	4	1.098986	+0.099289	+0.1000	+4.964	0.645
dilate	5	1	1.098986	+0.099289	+0.1000	+4.964	0.632
erode	1	4	0.979612	-0.020084	-0.0200	-1.004	0.487
erode	1	1	0.979612	-0.020084	-0.0200	-1.004	0.475
erode	2	4	0.959474	-0.040223	-0.0400	-2.011	0.494
erode	2	1	0.959474	-0.040223	-0.0400	-2.011	0.477
erode	5	4	0.899075	-0.100621	-0.1000	-5.031	0.458
erode	5	1	0.899075	-0.100621	-0.1000	-5.031	0.447
open	2	4	0.999247	-0.000449	+0.0000	-0.022	0.547
close	2	4	0.999265	-0.000432	+0.0000	-0.022	0.533

わかったこと

- ・ SDF をちょうど d だけ太らせる（痩せさせる）なら、Offset = d ÷ 升の大きさ。
- ・ Iterations は Dilate ・ Erode の量を変えない。
- ・ Open ・ Close は丸い形をほとんど変えない。細い突起や小さな穴を消すための処理。

次にやること

- ・ 角のある箱での Open ・ Close の効き方
- ・ Offset を升目より細かく（小数で）指定したとき

vdbreshapesdf の Open は箱の角を r より少し大きく丸める — Close も凸な箱を少しだけ削る

2026-09-24

何を作ったか

1 × 1 × 1 の箱を vdbfrompolygons (Voxel Size 0.01) で SDF にし、vdbreshapesdf の Open ・ Close を Offset 5 ・ 10 ・ 20 升 (r = 0.05 ・ 0.1 ・ 0.2) でかけた。面に戻す前の元の体積は 1.000000。

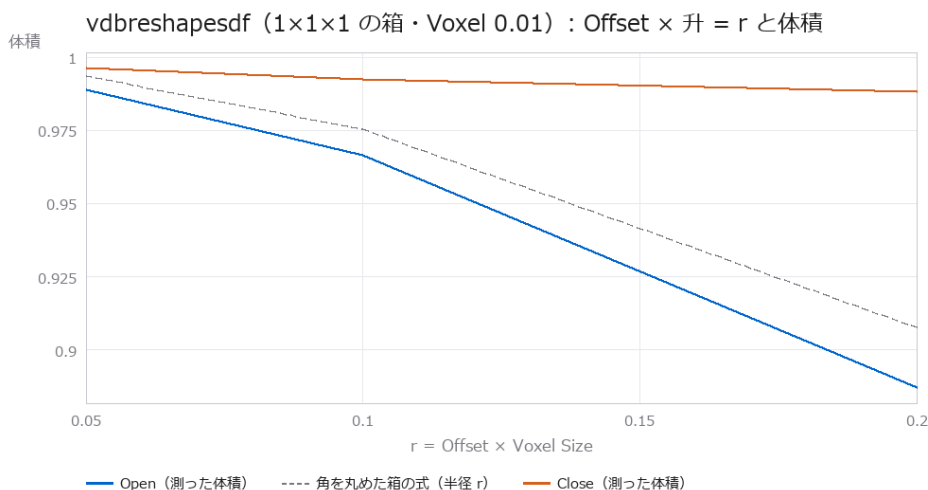
**Open は角を丸めたが、式の半径 r より大きく削れた。 **半径 r で角を丸めた箱の体積 (実験152 の式) に比べて、r = 0.05 で 0.9890 (式 0.9937) 、 r = 0.1 で 0.9666 (式 0.9756) 、 r = 0.2 で 0.8872 (式 0.9077) 。同じ体積になる丸みの半径は 0.067 ・ 0.117 ・ 0.223 (r の 1.33~1.11 倍。Offset が小さいほど倍率が大い) 。

Close は、へこみの無い箱を変えないはずだが、少し削った (r = 0.05 で -0.37% ・ r = 0.1 で -0.75% ・ r = 0.2 で -1.16%) 。Offset が SDF の帯 (既定 3 升) を超えるほど大きいと、太らせて戻す途中で角の情報が失われるとみられるが、確かめていない。

**実験195 の球では Open ・ Close はほぼ何もしなかった。 **角のある形でだけ、この差が出る。

**訂正 (実験198 ・ 199) : 上の「同じ体積の丸み半径」は、三つの面が集まる角の形も混ざった値だった。 **辺の断面に円を当てはめると、丸みは円弧で、半径は Offset × 升 + 約 3 升 (r = 0.05 ・ 0.1 ・ 0.2 で 0.075 ・ 0.130 ・ 0.232) 。「Offset が小さいほど倍率が大い」のは、この一定の足し分のため。帯の幅を変えても足し分は変わらなかった。

Operation ・ Offset と体積



Open (青) は式 (点線) より下。Close (橙) も 1 から少し下がる。

Operation	Offset (升)	r	体積	式	同じ体積の丸み半径	秒
(なし)	0	0	1.000000	1.000000	—	0.000
open	5	0.05	0.988964	0.993729	0.0666	0.732
open	10	0.1	0.966629	0.975587	0.1175	0.805

Operation	Offset (升)	r	体積	式	同じ体積の丸み半径	秒
open	20	0.2	0.887152	0.907705	0.2226	0.887
close	5	0.05	0.996274	1.000000	—	0.727
close	10	0.1	0.992465	1.000000	—	0.829
close	20	0.2	0.988423	1.000000	—	1.095

わかったこと

- Open で角を丸めると、Offset × 升より少し大きい丸みになる。辺の断面を測ると、足し分は倍率ではなく約 3 升で一定だった（実験198・199）。欲しい半径 R なら $\text{Offset} \approx R \div \text{升} - 3$ 。
- 凸な形に Close をかけても、少し削れる。Offset が大きいほど。
- 角を正確な半径で丸めたいなら polybevel（実験152）。

次にやること

- SDF の帯を広げたときの Close（実験197）
- Open の丸みの形（実験198）

SDF の帯を 25 升到げると、vdbreshapesdf の Close は箱をほとんど削らなくなる — Open の丸みはかえって大きくなる

2026-09-24

何を作ったか

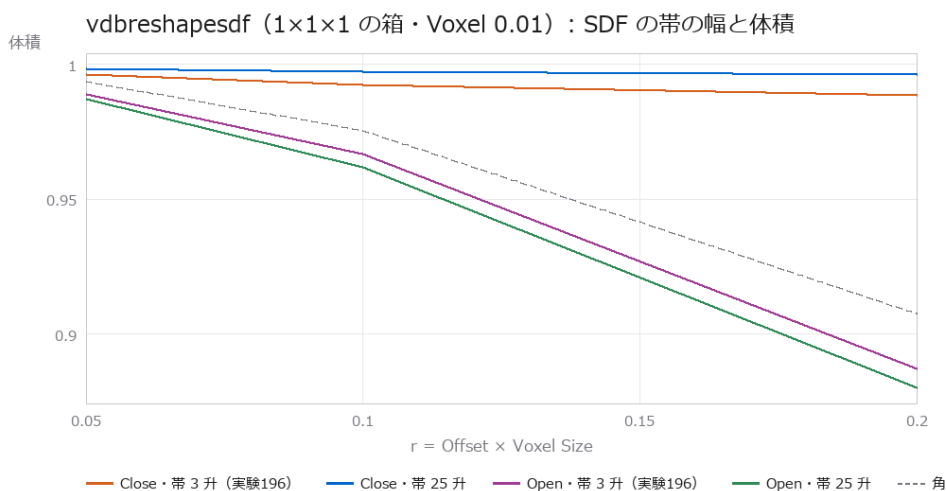
実験196 と同じ $1 \times 1 \times 1$ の箱 (Voxel Size 0.01) で、vdbfrompolygons の Exterior Band Voxels・Interior Band Voxels を既定の 3 から 25 に広げ、vdbreshapesdf の Open・Close を Offset 5・10・20 升到け直した。

**Close の削れは小さくなった。 **帯 3 升では -0.37%・-0.75%・-1.16%、帯 25 升では -0.18%・-0.27%・-0.40% (Offset 5・10・20 の順)。0 にはならず、Offset が大きいほど少し削れる。

**Open は、帯を広げるとかえって多く削れた。 **体積は帯 3 升で 0.9890・0.9666・0.8872、帯 25 升で 0.9870・0.9619・0.8802 (式 0.9937・0.9756・0.9077)。同じ体積になる丸みの半径は帯 25 升で 0.072・0.126・0.230 (r の 1.45~1.15 倍)。帯が狭いときは、帯の外の情報が無いことで削りが途中で止まっていたとみられるが、確かめていない。

時間は帯 25 升のほうがかかった (1 回 1.0~2.8 秒。帯 3 升では 0.7~1.1 秒)。

帯の幅・Operation・Offset と体積



Close は帯 25 升 (青) で 1 に近づく。Open は帯 25 升 (緑) のほうが式 (点線) から遠い。

帯 (升)	Operation	Offset (升)	r	体積	式	同じ体積の丸み半径	秒
3	open	5	0.05	0.988964	0.993729	0.0666	0.732
3	open	10	0.1	0.966629	0.975587	0.1175	0.805
3	open	20	0.2	0.887152	0.907705	0.2226	0.887
3	close	5	0.05	0.996274	1.000000	—	0.727
3	close	10	0.1	0.992465	1.000000	—	0.829
3	close	20	0.2	0.988423	1.000000	—	1.095

帯 (升)	Operation	Offset (升)	r	体積	式	同じ体積の丸み半径	秒
25	open	5	0.05	0.987034	0.993729	0.0723	1.031
25	open	10	0.1	0.961931	0.975587	0.1258	1.346
25	open	20	0.2	0.880163	0.907705	0.2299	1.749
25	close	5	0.05	0.998184	1.000000	—	1.108
25	close	10	0.1	0.997333	1.000000	—	1.589
25	close	20	0.2	0.996029	1.000000	—	2.833

わかったこと

- ・ Close で凸な形を削りたくないなら、SDF の帯を Offset より広く。それでも 0.2~0.4% は削れる。
- ・ Open の丸みは、帯を広げると r の 1.15~1.45 倍になる（小さい丸みほど倍率が大きい）。狙いの半径が要るなら polybevel（実験152）。
- ・ 帯を広げると時間が増える。

次にやること

- ・ Open の丸みの形（本当に円弧か）
- ・ vdbsmoothsdf と Open の違い

vdbreshapesdf の Open で丸めた辺の断面は円弧 ー 半径は Offset × 升より 2.5～3.2 升大きい

2026-09-24

何を作ったか

1 × 1 × 1 の箱を vdbfrompolygons (Voxel Size 0.01・帯は既定) で SDF にし、vdbreshapesdf の Open を Offset 5・10・20 升でかけて面に戻した。z = 0 の面の中で原点から 0～90 度に 721 本の光線を撃って当たった点を取り、辺の近くの点に円を当てはめた (最小二乗)。

**断面は円弧だった。 **当てはめの残差は RMS で 6.4e-05・3.4e-05・4.4e-05 (升の 0.6% 以下) 。円の中心は (0.5 - R, 0.5 - R) から 0.0017 以内で、両側の平らな面に接している。

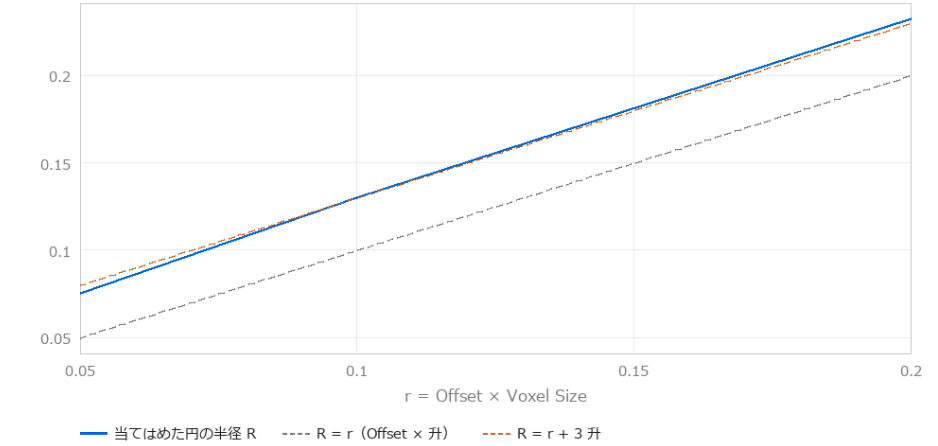
**半径は r (= Offset × 升) より大きかった。 **R = 0.0751・0.1303・0.2324 (r = 0.05・0.1・0.2) 。差は 2.5・3.0・3.2 升で、r によらずほぼ一定。倍率にすると 1.50・1.30・1.16 倍で、実験196 で「Offset が小さいほど倍率が大い」と見えたのは、この一定の足し分のためとみられる。足し分が帯の既定値 (3 升) に近いが、帯との関係は確かめていない。

**実験196 の体積から出した丸みの半径 (0.067・0.117・0.223) は、この R より小さい。 **体積には三つの面が集まる角の形も入るためとみられる (確かめていない) 。辺の半径は断面で測った。

時間は SDF にするところから面に戻すまで 1 回 0.14～0.29 秒 (光線の当たり判定は含まない) 。

Offset と、辺の断面に当てはめた円

vdbreshapesdf Open (1×1×1 の箱・Voxel 0.01) : 辺の断面に当てはめた円の半径 R



測った R (青) は R = r (灰の点線) より 2.5～3.2 升上をたどる。

Offset (升)	r	当てはめた点	R	R - r (升)	中心 x	接する点 x	残差 RMS	残差最大	対角の点 x	半径 r の円弧なら	秒
5	0.05	55	0.07515	2.51	0.42582	0.42485	6.4e-05	1.4e-04	0.47906	0.48536	0.135
10	0.1	103	0.13025	3.02	0.37147	0.36975	3.4e-05	7.6e-05	0.46357	0.47071	0.173

Offset (升)	r	当ては めた点	R	R - r (升)	中心 x	接する なら 0.5 - R	残差 RMS	残差 最大	対角の 点 x	半径 r の円弧 なら	秒
20	0.2	217	0.2323 5	3.23	0.2692 4	0.2676 5	4.4e-0 5	1.5e-0 4	0.4335 6	0.4414 2	0.294

わかったこと

- ・ Open は辺をきれいな円弧で丸める。面とのつなぎ目に段差はない。
- ・ 半径は Offset × 升 + 約 3 升。半径 R が欲しいなら Offset ÷ R ÷ 升 - 3 (升 0.01 ・ 帯既定の箱で) 。
- ・ 丸みの半径は体積ではなく断面で測る。

次にやること

- ・ 帯の幅を変えたときの R - r
- ・ 三つの面が集まる角は球か

Open で丸めた辺の半径が約 3 升大きいのは、SDF の帯のせいではない — 帯 3~25 升で足し分は変わらない

2026-09-24

何を作ったか

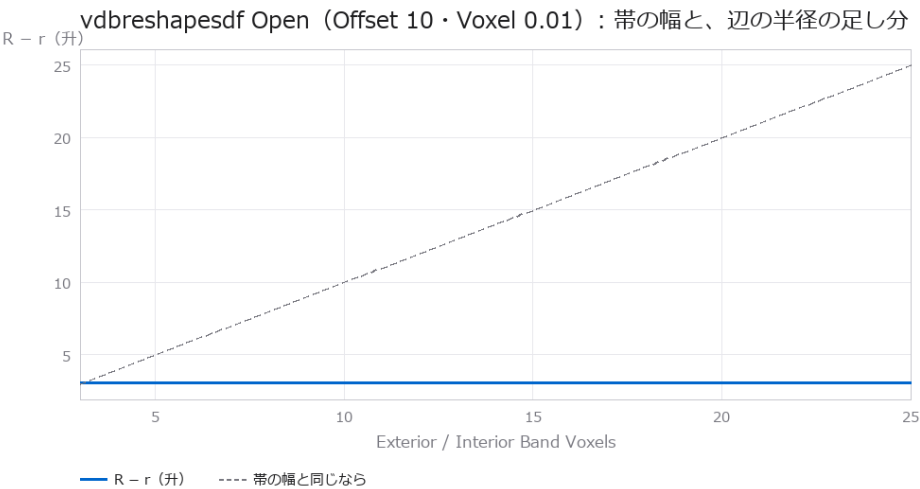
実験198 で、vdbreshap sdf の Open で丸めた辺の半径は Offset × 升より約 3 升大きかった。帯の既定値（3 升）と近いので、vdbfrompolygons の Exterior / Interior Band Voxels を 3・6・12・25 升に変え、Open（Offset 10 升・Voxel Size 0.01・1 × 1 × 1 の箱）の辺の断面を実験198 と同じやり方で測った。

**足し分 $R - r$ は、帯の幅によらず 3.02~3.03 升だった。 ** $R = 0.13025 \cdot 0.13028 \cdot 0.13029 \cdot 0.13029$ 。帯のせいではない。何が 3 升を足しているかは確かめていない。

体積は帯 3 升と 6 升で違い、6 升以上は同じだった（0.966629・0.961948・0.961931・0.961931）。辺の半径は変わらないので、実験197 で見た帯による体積の違いは、辺以外（三つの面が集まる角など）から来ているとみられる。確かめていない。

時間は帯が広いほどかかった（0.21 秒 → 0.77 秒）。

帯の幅と、辺の断面の円



足し分（青）は帯の幅（点線）と関係なく 3 升で平ら。

帯（升）	Offset（升）	当てはめた点	R	R - r（升）	中心 x	接するなら 0.5 - R	残差 RMS	体積	秒
3	10	103	0.13025	3.025	0.37147	0.36975	3.4e-05	0.966629	0.210
6	10	109	0.13028	3.028	0.37079	0.36972	8.9e-05	0.961948	0.272
12	10	109	0.13029	3.029	0.37078	0.36971	8.9e-05	0.961931	0.430

帯 (升)	Offset (升)	当てはめた点	R	R - r (升)	中心 x	接するなら 0.5 - R	残差 RMS	体積	秒
25	10	109	0.13029	3.029	0.37078	0.36971	8.9e-05	0.961931	0.768

わかったこと

- ・ Open の辺の半径は、帯を広げても変わらない。Offset $\div$ 升 - 3 は帯によらず使える（升 0.01 の箱で）。
- ・ 帯は 6 升あれば体積も変わらなくなった。それ以上広げると時間だけ増える。

次にやること

- ・ 升の大きさを変えたときの足し分（3 升か、一定の長さか）
- ・ 三つの面が集まる角は球か